

Efficient Deep Learning of Robust Policies from MPC using Imitation and Tube-Guided Data Augmentation

Andrea Tagliabue and Jonathan P. How

Abstract—Imitation Learning (IL) can generate computationally efficient policies from demonstrations provided by Model Predictive Control (MPC). However, IL methods often require extensive data-collection and training-efforts, limiting changes to the policy if the task changes, and they produce policies with limited robustness to new disturbances. In this work, we propose an IL strategy to *efficiently* compress a computationally expensive MPC into a deep neural network policy that is *robust* to previously unseen disturbances. By using a robust variant of the MPC, called Robust Tube MPC, and leveraging properties from the controller, we introduce computationally-efficient data augmentation methods that enable a significant reduction of the number of MPC demonstrations and training efforts required to generate a robust policy. Our approach opens the possibility of *zero-shot* transfer of a policy trained from a single MPC demonstration collected in a nominal domain, such as a simulation or a robot in a lab/controlled environment, to a new domain with previously unseen bounded model errors/perturbations. Numerical evaluations performed using linear and nonlinear MPC for agile flight on a multirotor show that our method outperforms strategies commonly employed in IL (such as Dataset-Aggregation (DAGger) and Domain Randomization (DR)) in terms of demonstration-efficiency, training time, and robustness to perturbations unseen during training. Experimental evaluations validate the efficiency and real-world robustness.

Index Terms—Imitation Learning; Data Augmentation; Robust Tube Model Predictive Control; Aerial Robotics.

SUPPLEMENTARY MATERIAL

Video: <https://youtu.be/-uiarBY1STU>

I. INTRODUCTION

Model Predictive Control (MPC) [1], [2] enables impressive performance on complex, agile robots [3]–[8]. However, its computational cost often limits the opportunities for onboard, real-time deployment [9] on platforms with limited computation [10], [11], or diverts critical computing power needed by other components governing the autonomous system. Recent works have mitigated MPC’s computational requirements by relying on computationally efficient deep neural network (DNN) policies that are trained to imitate task-relevant demonstrations generated by MPC in an offline training phase. Such demonstrations are generally collected via Imitation Learning (IL) [12]–[14], where the MPC acts as an expert that provides demonstrations, and the DNN policy is treated as a student, trained via supervised learning.

A common issue in existing IL methods (e.g., Behavior Cloning (BC) [15]–[17], Dataset-Aggregation (DAGger) [18])

The authors are with the Department of Aeronautics and Astronautics, Massachusetts Institute of Technology. {atagliab, jhow}@mit.edu.

We thank Prof. Michael Everett, Dr. Dong-Ki Kim, Tong Zhao and Prof. Donggun Lee, Kota Kondo, and Xiaoyi Cai for feedback and discussions. Work funded by the AFOSR MURI FA9550-19-1-0386.

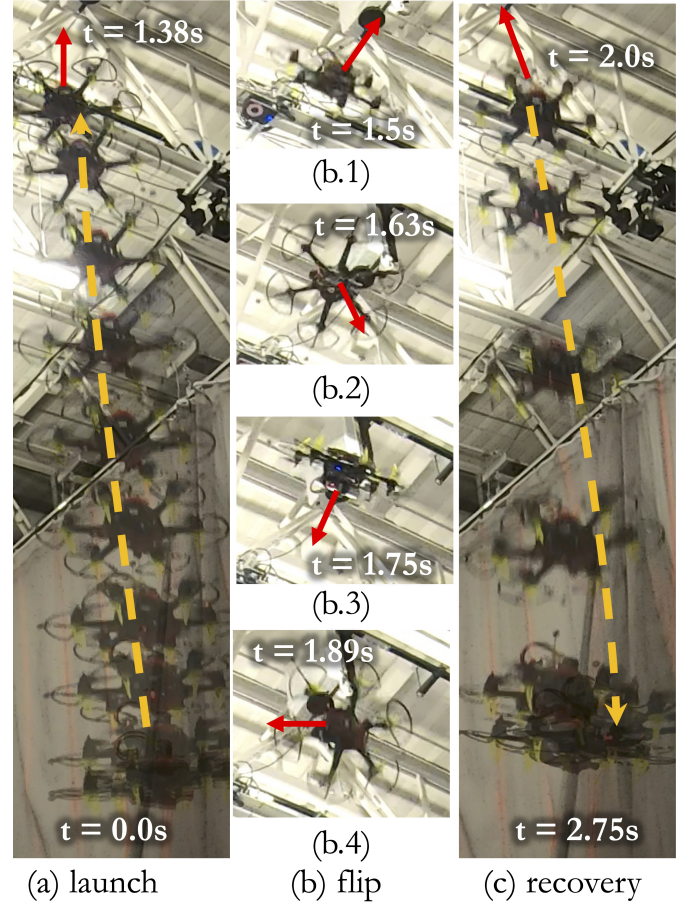


Fig. 1: Time-lapse of a multirotor performing a flip using a DNN policy learned via the proposed approach. The policy is learned offboard efficiently (in only 100 s of training time), and deployed onboard (NVIDIA Jetson TX2, CPU), tested at up to 500 Hz, with an average inference time of 15 μ s. (a) Upwards acceleration phase (red arrow: thrust vector, yellow arrow: trajectory). (b) 360° rotation around the body x -axis in ~ 0.5 s. (c) Deceleration phase.

is that they require to collect a relatively large number of MPC demonstrations, even for a single task like tracking a specific trajectory. This sample-inefficiency introduces significant challenges: (i) it necessitates a substantial number of queries to the resource-intensive MPC expert, requiring expensive training equipment; (ii) it hinders learning from very high-dimensional MPC experts; (iii) it results in a considerable volume of queries to the training environment, limiting data collection in computationally intensive simulations or demanding numerous hours of real-time demonstrations on a physical robot, which is impractical. Moreover, this approach complicates updating the policy when the MPC expert undergoes changes due to (iv) tuning or (v) model updates, or when (vi) performing new tasks

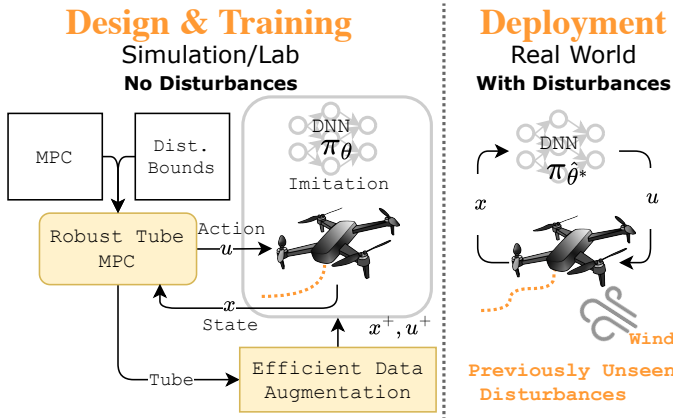


Fig. 2: Overview of the approach proposed to generate a DNN-based policy π_θ from a computationally expensive MPC in a data and compute-efficient way. We do so by generating a Robust Tube MPC (RTMPC) using bounds of the disturbances encountered in the deployment domain. We use properties of the tube to derive a computationally efficient data augmentation (DA) strategy that generates extra state-action pairs (x^+, u^+) , obtaining π_{θ^*} via IL. Our approach enables zero-shot transfer from a single demonstration collected in simulation (*sim2real*) or a controlled environment (lab, factory, *lab2real*).

is required, such as tracking different sets of trajectories. These aspects can be particularly critical in fields such as chemical and process control, where state dimension can reach 252 and planning horizon of 140 nodes [9], [19], [20], resulting in computationally intractable online optimization problems and offline policy generation procedures.

One of the causes for such demonstration-inefficiency is the need to take into account and correct for the compounding of errors (*covariate or distribution shifts*) in the learned policy [18], which may otherwise create catastrophic consequences [15]. These distribution shifts can be caused by: (a) mismatches, e.g., due to modeling errors, between the simulator used to collect demonstrations and the deployment domain (i.e., *sim2real* gap); (b) learning errors in the policy; or (c) model changes or disturbances that may not be present in a controlled training environment (lab/factory when training on a real robot), but that do appear during deployment in the real-world (i.e., *lab2real* gap). Approaches employed to compensate for these *gaps* and generate a robust policy, such as Domain Randomization (DR) [21], [22], introduce additional challenges, such as the need to apply disturbances or model changes during training. Data and computational-efficiency challenges in IL can be mitigated by DA strategies, based on augmenting the training data with extra input-output samples *efficiently-generated* from the collected demonstrations [15], [17], [23]–[25]. However, existing methods for MPC [23]–[25] do not explicitly account for uncertainties, not only in the way the demonstration are generated, but more importantly in the way the samples are generated, resulting in policies with limited robustness to uncertainties.

A. Efficient, Robust IL from MPC via Sampling Augmentation

In this work, we address the problem of generating a robust DNN policy from MPC in a demonstration and computationally efficient manner by designing a computationally-efficient DA strategy that systematically compensates for the effects of covariate shifts that might be encountered during real-world deployment. Our approach, named Sampling Augmentation (SA) and depicted in Fig. 2, relies on a prior model of

the perturbations/uncertainties encountered in a deployment domain, which is used to generate a robust version of the given MPC, called RTMPC, to collect demonstrations and to guide the DA strategy. The key idea behind this DA strategy consists in observing that the RTMPC framework provides: (a) information on the states that the robot may visit when subject to uncertainty. This is represented by a *tube* that contains the collected demonstration; the tube can be used to identify/generate extra relevant states for DA; and (b) an *ancillary controller* that maintains the robot inside the tube regardless of the realization of uncertainties; this controller can be used to generate extra actions. To numerically and experimentally validate our approach, we tailor SA to the task of efficiently learning robust policies for agile flight on a multirotor. First, we demonstrate in experiments trajectory tracking capabilities with a policy learned from a linear trajectory tracking RTMPC. The policy is learned from a *single* demonstration collected in simulation or directly on the real robot, and it is robust to previously-unseen wind disturbances. Second, we demonstrate the ability to generate a policy from a go-to-goal-state nonlinear RTMPC capable of performing acrobatic maneuvers, such as a 360 degrees flip. These maneuvers are performed under real-world uncertainties, using a policy obtained from only *two* demonstrations and in less than 100s of training time.

B. Related Work

Explicit MPC [1] approximates linear MPC by pre-computing a policy (look-up table or DNNs [26]) offline, partitioning the state space. However, its memory and computational complexity grow exponentially with the number of constraints. Our work addresses this by training efficient DNN policies [9] through task-relevant demonstrations and IL, focusing on the most relevant parts of the policy input space and learning from MPCs with nonlinear models.

IL from MPC. Imitation-learned policies from MPC are widely used in robotics. Close to our work, [12] learns to perform acrobatic maneuvers with a quadrotor from MPC using DAgger combined with DR, by collecting 150 demonstrations in simulation. Ref. [27] uses DAgger combined with an MPC based on differential dynamic programming (DDP) [28] for agile off-road autonomous driving using about 24 laps¹ around their racetrack for the first DAgger iteration. These examples show the *performance* that can be achieved using IL from MPC, but they also highlight that current methods require a large number of interactions with the MPC and the training environment, resulting in longer training times or complex data collection procedures, as summarized in Table I.

Robustness in IL. Robustness in IL is needed to compensate for the distribution shifts caused by the *sim2real* or *lab2real* (i.e., when collecting demonstrations on the real robot in a controlled environment and then deploying in the real world) transfers. Robustness to these types of *shifts* is achieved by modifying the training domain so that its dynamics match the ones encountered in the deployment domain [21], [32]. An extremely effective method is DR [21], which applies random model errors/disturbances, sampled from a predefined set of

¹Obtained using Table 2 in [27], considering 6000 observation/action pairs sampled at 50 Hz while racing on a 30 m long racetrack with an average speed of 6 m/s.

TABLE I: Strategies for policy learning from model-based planners/controllers. Our work is the only method that enables efficient learning of policies that account for uncertainties.

Method	Explicitly Accounts for uncertainties	Data-efficient training	Compute-efficient training	Allows both off/on-policy data collection and real robot	Demonstrations State ≥ 10 from both sim. and under-actuated	Real-world and agile deployment
BC [15]	No	No	No	n.a.	Yes	No
Dagger [13], [27]	No	No	No	n.a.	Yes	Yes
DR [21], [22]	Yes	No	No	Yes	Yes	Yes
GPS [23], [29]	No	Yes	Yes	n.a.	No	No
MPC-Net [30]	No	Yes	Yes	No	Yes	No
LAG-ROS [31]	Yes	n.a.	No	No	No	No
[24] (DA)	No	No	Yes	No	No	No
[25] (DA)	No	Yes	Yes	No	No	No
SA (proposed)	Yes	Yes	Yes	Yes	Yes	Yes

possible perturbations, during data collection in simulation. An alternative avenue relies on modifying the actions of the expert to ensure that the state distribution visited at training time matches the one encountered at deployment time, such as in DART [33]. Although effective, these approaches require many demonstrations/interactions with the environment in order to take into account all the possible instantiations of model errors/disturbances that might be encountered in the target domain, limiting the opportunities for *lab2real* transfers, or increasing the data collection effort when training in simulation. Our work will exploit extra information available to the MPC to reduce the number of MPC/environment interactions.

Data Augmentation for Efficient/Robust IL. Guided Policy Search (GPS) [14], [23], [29], [30], [34], introduced first the idea to use trajectories from model-based planners, including MPC, to generate state-action samples (guiding samples) for improved sample efficiency in policy learning. Specifically, Ref. [23] leveraged an iterative linear quadratic regulator (iLQR) [5] expert to generate guiding samples around the optimal trajectory found by the controller. Similarly, the authors in [30] observe that adding extra states and actions sampled from the neighborhood of the optimal solution found by the iLQR expert can reduce the number of demonstrations required to learn a policy when using DAgger. However, while GPS methods are in general more sample-efficient than IL, the nominal plans and the distribution of guiding-samples they generate do not *explicitly* account for model and environment uncertainties, resulting in policies with limited robustness. Ref. [34] for example, demonstrates in simulation robustness to up to 3.3% in weight perturbations of a multirotor, while our approach demonstrates robustness to perturbations up to 30%. Our work leverages a robust variant of MPC called RTMPC [35], [36], to provide robust demonstrations and a DA strategy that accounts for the effects of uncertainties. Specifically, the DA strategy is obtained by using an outer-approximation of the robust control invariant set (*tube*) as a support of the sampling distribution, ensuring that the guiding samples produce robust policies. This idea is related to the recent LAG-ROS framework [31], which provides a learning-based method to compress a global planner in a DNN by extracting relevant information from the robust tube. LAG-ROS emphasizes the importance of nonlinear contraction-based controllers (e.g., CV-STEM [37]) to obtain robustness and stability guarantees. Our contribution emphasizes instead minimal requirements - namely a tube and an *efficient* DA strategy - to achieve demonstration-efficiency and robustness to real-world conditions. By decoupling these aspects from the need for complex control strategies, our work greatly simplifies the controller design. Additionally, different from LAG-ROS,

the DA procedures presented in our work do not require solving a large optimization problem for every extra state-action sample generated (achieving computational efficiency during training) and can additionally leverage interactive experts (e.g., DAgger) to trade off the number of interactions with the environment with the number of extra samples from DA (further improving training efficiency).

Recent work [24], [25] exploited local approximations of the solutions found when solving the nonlinear program (NLP) associated with MPC to efficiently generate extra state-actions samples for DA in policy learning. Similar to our work, [24] uses a parametric sensitivity-based approximation of the solution to efficiently generate extra states and actions. Different from our work, however, their method proposes sampling of the entire feasible state space to learn a policy, while our work focuses instead on task-relevant demonstrations, a more computationally and data-efficient solution. The recently presented extension [25] solves this issue by leveraging interactive experts (e.g., DAgger). However, both [24], [25] do not *explicitly* account for the effects of uncertainties, neither in the design of the expert, nor in the way that extra states are generated, resulting in policies with limited robustness. Thanks to our robust expert, our approach not only accounts for uncertainties during demonstration collection and in the distribution of samples for DA, but it can additionally account for the errors introduced in the DA procedure by further constraint tightening and updating the tube size. Additionally, thanks to the strong prior on the state distribution under uncertainty produced by the tube in RTMPC, our DA strategy can quickly cover the task-relevant parts of the state space, obtaining demonstration-efficiency. Last, unlike prior work, we experimentally validate our approach, demonstrating it on a system whose models has a large state size (state size 8 and 10), whereas previous work focuses on lower-dimensional systems (state size 2) and only in simulation.

Robustness and Computational Challenges in MPC for Agile Flight. MPC has been widely employed in the aerial robotics community [38], enabling impressive performance in trajectory tracking and minimum-time planning for agile flights, and particularly in drone racing [39]–[41]. However, the authors of [39] highlight that one of the biggest drawbacks of MPC is in its required computational resources, limiting its deployment on platforms with a small computational budget. In addition, they highlight that their MPC tends to fail when subject to a large external force disturbance or model errors. Our work is motivated by these findings and employs robust variants of MPC that explicitly account for uncertainties, such as disturbances and model errors, while reducing the computational complexity of MPC. Impressive agile flight has also been achieved by MPC with models learned offline [42], [43] or online [44], [45], or with MPC combined with non-parametric adaptation laws [46]. While our work does not directly tackle the numerous challenges associated with adaptation and model learning in MPC, we highlight that our approach can benefit these fields, as RTMPC can explicitly account for uncertainties in learned models and can account for the dynamics introduced by adaptation laws [4], reducing the constraint violations observed in [46].

C. Contributions

This article extends our prior conference paper [47], where the focus was on efficiently generating robust trajectory tracking policies from a *linear* MPC. In this new work, we additionally provide a strategy to generate a DNN policy to reach a desired state using a *nonlinear* MPC expert, presenting a new methodology that can be used to perform DA in a computationally-efficient way. This extension is non-trivial, as the *ancillary controller* in the nonlinear RTMPC framework [36] requires, unlike the linear case, expensive computations to generate extra actions for DA, resulting in long training times when performing DA. This new work solves the computational-efficiency issues in the ancillary controller of nonlinear RTMPC by generating a sensitivity-based approximation of the ancillary controller that is used to more efficiently compute the actions corresponding to extra state samples for DA. While sensitivity-based approximations of traditional MPC were first explored in recent work [24], this work extends [24] not only by (1) learning from demonstrations from a controller [36] whose nominal plans account for uncertainties, but additionally (1.1) proposes a sampling strategy based on the tube in [36], used as support of the sampling distribution, rather than considering arbitrary neighborhoods of the state space [24], achieving robustness and data-efficiency; (1.2) leverages both on-policy (Dagger) and off-policy (BC) data collection methods, enabling trade-offs in terms of performance of the learned policy versus ease of data collection; (1.3) presents a policy fine-tuning procedure to minimize the impact on performance introduced by the sensitivity-based approximation; (1.4) leverages further constraint tightening (e.g., makes constraints more conservative) in [36] to account for errors introduced by the approximate DA strategy, ensuring robustness. Additionally, this work presents (2) a formulation of nonlinear RTMPC for acrobatic flights on multirotors; (3) numerical comparison to IL baselines; (4) numerical comparison of different tube-sampling strategies; (5) new real-world experiments with policies that leverage nonlinear models.

In summary, our work presents the following **contributions**:

- A procedure to *efficiently* learn *robust* policies from MPC. Our procedure is: 1) *demonstration-efficient*, as it requires a small number of queries to the training environment, resulting in a method that enables learning from a single MPC demonstration collected in simulation or on the real robot; 2) *training-efficient*, as it reduces the number of computationally expensive queries to the computationally expensive MPC expert using a computationally efficient DA strategy; 3) *generalizable*, as it produces policies robust to disturbances not experienced during training.
- We generalize the demonstration-efficient policy learning strategy proposed in our previous conference paper [47] with the ability to efficiently learn robust and generalizable policies from variants of MPC that use nonlinear models.
- Extensive simulations and comparisons with state-of-the-art IL methods and robustification strategies.
- Experimental evaluation on the challenging task of trajectory tracking and acrobatic maneuvers on a multirotor, presenting the first instance of sim2real transfer of a policy trained after a single demonstration, and robust to previously unseen real-world uncertainties.

II. PROBLEM STATEMENT

This part describes the problem of learning a robust policy in a demonstration and computationally efficient way by imitating an MPC expert demonstrator. Robustness and efficiency are determined by the ability to design an IL procedure that can compensate for the covariate shifts induced by uncertainties encountered during real-world deployment while collecting demonstrations in a domain (the training domain) that presents only a subset of those uncertainty realizations. Our problem statement follows the one of robust IL (e.g., DART [33]), modified to use deterministic policies/experts and to account for the differences in uncertainties encountered in deployment and training domains. Additionally, we present a common approach employed to address the covariate shift issues caused by uncertainties, DR, highlighting its limitations.

A. Assumptions and Notation

System Dynamics. We assume the dynamics of the real system are Markovian and stochastic [48], and can be described by a twice continuously differentiable function $f(\cdot)$:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_t, \quad (1)$$

where $\mathbf{x}_t \in \mathbb{X} \subseteq \mathbb{R}^{n_x}$ represents the state, $\mathbf{u}_t \in \mathbb{U} \subseteq \mathbb{R}^{n_u}$ the control input in the compact subsets $\mathbb{X}$, $\mathbb{U}$. $\mathbf{w}_t \in \mathbb{W}_{\mathcal{E}} \subset \mathbb{R}^{n_x}$ is an unknown state perturbation, belonging to a compact convex set $\mathbb{W}_{\mathcal{E}}$ containing the origin. Stochasticity in Eq. (1) is introduced by $\mathbf{w}_t$, sampled from a probability distribution having support $\mathbb{W}_{\mathcal{E}}$, under a (possibly unknown) probability density function, capturing the effects of noise, approximation errors in the learned policy, model changes, and other disturbances acting on the system during training or under real-world conditions at deployment.

Sim2Real and Lab2Real Transfer Setup. Three different environments/domains $\mathcal{E}$ are considered: a training domain based on a simulation $\mathcal{S}_{\text{sim}}$ (where $\mathcal{S}$ denotes *source*), a training domain based on the real robot in a controlled lab environment $\mathcal{S}_{\text{lab}}$, and a deployment domain $\mathcal{T}$ (*target*). Mathematically, the three domains differ in their transition probabilities. In all the domains, we do not assume knowledge on density function from which $\mathbf{w}_t$ is sampled, but we assume available prior knowledge of $\mathbb{W}_{\mathcal{T}}$, the support of the distribution (e.g, worst-case uncertainty realization) at deployment. This is a common assumption in robust control [35], [36], where such knowledge can come from historical data, regulatory requirements, or can be assumed to match the physical limits of the robot. Additionally, we assume $\mathbb{W}_{\mathcal{S}_{\text{lab}}} \subset \mathbb{W}_{\mathcal{T}}$ and $\mathbb{W}_{\mathcal{S}_{\text{sim}}} \subset \mathbb{W}_{\mathcal{T}}$, representing the fact that training is usually performed in simulation or in a controlled/lab environment under some nominal model errors/disturbances, while at deployment a larger set of perturbations can be encountered. Note that for convenience we use $\mathcal{S}$ to denote both $\mathcal{S}_{\text{sim}}$ and $\mathcal{S}_{\text{lab}}$.

Tracking MPC Expert. We consider a tracking MPC expert demonstrator that plans along an $N + 1$ -steps horizon. The expert is given the current state $\mathbf{x}_t$, and $\mathbf{X}_t^{\text{des}} \in \mathbb{X}_{N_{\text{des}}}^{\text{des}} := \{\mathbf{x}_{0|t}^{\text{des}}, \dots, \mathbf{x}_{N_{\text{des}}|t}^{\text{des}} | \mathbf{x}_i^{\text{des}} \in \mathbb{R}^{n_x}\}$, representing a desired state to be reached, or a state trajectory to be followed. Then, the MPC

expert generates control actions by solving an Optimal Control (OC) problem of the form:

$$\begin{aligned} \bar{\mathbf{X}}_t^*, \bar{\mathbf{U}}_t^* &= \underset{\bar{\mathbf{X}}_t, \bar{\mathbf{U}}_t}{\operatorname{argmin}} J_N(\bar{\mathbf{X}}_t, \bar{\mathbf{U}}_t, \mathbf{X}_t^{\text{des}}) \\ \text{subject to } &\bar{\mathbf{x}}_{0|t} = \mathbf{x}_t, \\ &\bar{\mathbf{x}}_{i+1|t} = f(\bar{\mathbf{x}}_{i|t}, \bar{\mathbf{u}}_{i|t}), \\ &\bar{\mathbf{x}}_{i|t} \in \mathbb{X}, \bar{\mathbf{u}}_{i|t} \in \mathbb{U}, \\ &i = 0, \dots, N-1. \end{aligned} \quad (2)$$

where J_N represents the cost to be minimized (where N denotes the dependency on the planning horizon), and $\bar{\mathbf{X}}_t = \{\bar{\mathbf{x}}_{0|t}, \dots, \bar{\mathbf{x}}_{N|t}\}$ and $\bar{\mathbf{U}}_t = \{\bar{\mathbf{u}}_{0|t}, \dots, \bar{\mathbf{u}}_{N-1|t}\}$ are sequences of states and actions along the planning horizon, where the notation $\bar{\mathbf{x}}_{i|t}$ indicates the planned state at the future time $t+i$, as planned at the current time t . At every timestep t , given $\mathbf{x}_t$, the control input applied to the real system is the first element of $\bar{\mathbf{U}}_t^*$, resulting in an implicit deterministic control law (policy) that we denote as $\pi_{\theta^*} : \mathbb{X} \times \mathbb{X}^{\text{des}} \rightarrow \mathbb{U}$.

DNN Student Policy. As for the MPC expert, we model the DNN student policy as a deterministic policy π_{θ} , with parameters θ , that does not necessarily belong to the same policy class as the expert. When considering trajectory tracking tasks, the policy takes as input the current state and the desired reference trajectory segment, $\pi_{\theta} : \mathbb{X} \times \mathbb{X}_{N+1}^{\text{des}} \rightarrow \mathbb{U}$. When considering the task of reaching a goal state, the policy takes as input the current state, the desired goal state and the current timestep $t \in \mathbb{I}_{\geq 0}$, $\pi_{\theta} : \mathbb{X} \times \mathbb{X}_0^{\text{des}} \times \mathbb{I}_{\geq 0} \rightarrow \mathbb{U}$.

Transition Probabilities. We denote the state transition probability under π_{θ} in a domain $\mathcal{E}$ for a given goal-reaching or trajectory tracking task as $p_{\pi_{\theta}, \mathcal{E}}(\mathbf{x}_{t+1}|\mathbf{x}_t)$. The probability of collecting a T -(state, action) pairs trajectory $\xi = \{(\mathbf{x}_t, \mathbf{u}_t)_{t=0}^{T-1}\}$, given a policy π_{θ} , depends on the deployment environment $\mathcal{E}$:

$$p(\xi|\pi_{\theta}, \mathcal{E}) = p(\mathbf{x}_0) \prod_{t=0}^{T-1} p_{\pi_{\theta}, \mathcal{E}}(\mathbf{x}_{t+1}|\mathbf{x}_t), \quad (3)$$

where $p(\mathbf{x}_0)$ represents the initial state distribution.

B. Robust Imitation Learning Objective

The objective of robust IL, following [33], is to find parameters θ of π_{θ} that minimize a distance metric $\mathcal{L}(\theta, \theta^*|\xi)$ from the MPC expert π_{θ^*} :

$$\hat{\theta}^* = \arg \min_{\theta} \mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{T})} \mathcal{L}(\theta, \theta^*|\xi). \quad (4)$$

This metric captures the differences between the actions generated by the expert π_{θ^*} and the action produced by the student π_{θ} across the distribution of trajectories induced by the student policy π_{θ} in the perturbed domain $\mathcal{T}$, as denoted by $p(\xi|\pi_{\theta}, \mathcal{T})$. The distance metric considered in this work is the Mean Squared Error (MSE) loss:

$$\mathcal{L}(\theta, \theta^*|\xi) = \frac{1}{T} \sum_{t=0}^{T-1} \|\pi_{\theta}(\mathbf{x}_t^{\text{in}}) - \pi_{\theta^*}(\mathbf{x}_t, \mathbf{X}_t^{\text{des}})\|_2^2. \quad (5)$$

where $\mathbf{x}_t^{\text{in}} = \{\mathbf{x}_t, \mathbf{X}_t^{\text{des}}\}$ for trajectory tracking tasks, and $\mathbf{x}_t^{\text{in}} = \{\mathbf{x}_t, \mathbf{X}_t^{\text{des}}, t\}$ for go-to-goal-state tasks.

Covariate Shift due to Sim2real and Lab2real Transfer. Because in practice we do not have access to the target

environment, the goal of Robust IL is to try to solve Eq. (4) by finding an approximation of the optimal policy parameters θ^* using data from the source environment:

$$\hat{\theta}^* = \arg \min_{\theta} \mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{S})} \mathcal{L}(\theta, \theta^*|\xi). \quad (6)$$

The way this minimization is solved depends on the chosen IL algorithm. The performance of the learned policy in the target and source domains can be related via:

$$\begin{aligned} \mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{T})} \mathcal{L}(\theta, \theta^*|\xi) &= \\ &\underbrace{\mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{T})} \mathcal{L}(\theta, \theta^*|\xi) - \mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{S})} \mathcal{L}(\theta, \theta^*|\xi)}_{\text{covariate shift due to transfer}} \\ &\quad + \underbrace{\mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{S})} \mathcal{L}(\theta, \theta^*|\xi)}_{\text{IL objective}}, \end{aligned} \quad (7)$$

which clearly shows the presence of a covariate shift induced by the transfer. The last term corresponds to the objective minimized by performing IL in $\mathcal{S}$. Attempting to solve Eq. (4) by directly optimizing Eq. (6) (e.g., via BC [15]) offers no assurances of finding a policy with good performance in $\mathcal{T}$.

C. Shift Compensation via Domain Randomization.

A well-known strategy to compensate for the effects of covariate shifts between source and target domain is DR [21], which modifies the transition probabilities of the source $\mathcal{S}$ by trying to ensure that the trajectory distribution in the modified training domain $\mathcal{S}_{\text{DR}}$ matches the one encountered in the target domain: $p(\xi|\pi_{\theta}, \mathcal{S}_{\text{DR}}) \approx p(\xi|\pi_{\theta}, \mathcal{T})$. This is done by applying perturbations to the robot during demonstration collection, sampling perturbations $\mathbf{w} \in \mathbb{W}_{\text{DR}}$ according to some knowledge/hypotheses on their distribution $p_{\mathcal{T}}(\mathbf{w})$ in the target domain [21], obtaining the perturbed trajectory distribution $p(\xi|\pi_{\theta}, \mathcal{S}, \mathbf{w})$. The minimization of Eq. (4) can then be approximately performed by minimizing instead:

$$\mathbb{E}_{p_{\mathcal{T}}(\mathbf{w})} [\mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{S}, \mathbf{w})} \mathcal{L}(\theta, \theta^*|\xi)]. \quad (8)$$

This approach, however, requires the ability to apply disturbances/model changes to the system, which may be impractical e.g., in the *lab2real* setting, and may require a large number of demonstrations due to the need to sample enough state perturbations $\mathbf{w}$.

III. EFFICIENT LEARNING FROM LINEAR RTMPC

In this Section, we present the strategy to efficiently learn robust policies from MPC when the system dynamics in Eq. (1) can be well approximated by a linear model of the form:

$$\mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t + \mathbf{w}_t. \quad (9)$$

First, we present the Robust Tube variant of linear MPC, RTMPC, that we employ to collect demonstrations (Section III-A). Then, we present a strategy that leverages information available from the RTMPC expert to compensate for the covariate shifts caused by uncertainties and mismatches between the training and deployment domains (Section III-B). Our strategy is based on a DA procedure that can be combined with different IL methods (on-policy, such as DAgger [18], and off-policy, such as BC, [15]) for improved efficiency/robustness in the policy learning procedure. The RTMPC expert is based on [35] but with the objective function modified to track desired

trajectories, as trajectory-tracking tasks will be the focus of the experimental evaluation of policies learned from this controller (Section VI).

A. Trajectory Tracking RTMPC Expert Formulation

RTMPC is a type of robust MPC that regulates the system in Eq. (9) while ensuring satisfaction of the state and actuation constraints $\mathbb{X}, \mathbb{U}$ regardless of the disturbances $\mathbf{w} \in \mathbb{W}_{\mathcal{T}}$.

Mathematical Preliminaries. Let $\mathbb{A} \subset \mathbb{R}^n$ and $\mathbb{B} \subset \mathbb{R}^n$ be convex polytopes, and let $\mathbf{C} \in \mathbb{R}^{m \times n}$. Then we define:

- a) Linear mapping: $\mathbf{C}\mathbb{A} := \{\mathbf{C}\mathbf{a} \in \mathbb{R}^m \mid \mathbf{a} \in \mathbb{A}\}$
- b) Minkowski sum: $\mathbb{A} \oplus \mathbb{B} := \{\mathbf{a} + \mathbf{b} \in \mathbb{R}^n \mid \mathbf{a} \in \mathbb{A}, \mathbf{b} \in \mathbb{B}\}$
- c) Pontryagin diff.: $\mathbb{A} \ominus \mathbb{B} := \{\mathbf{c} \in \mathbb{R}^n \mid \mathbf{c} + \mathbf{b} \in \mathbb{A}, \forall \mathbf{b} \in \mathbb{B}\}$.

Optimization Problem. At each time step t , trajectory tracking RTMPC receives the current robot state $\mathbf{x}_t$ and a desired trajectory $\mathbf{X}_t^{\text{des}} = \{\mathbf{x}_{0|t}^{\text{des}}, \dots, \mathbf{x}_{N|t}^{\text{des}}\}$ spanning $N + 1$ steps as input. It then computes a sequence of reference (“safe”) states $\bar{\mathbf{X}}_t = \{\bar{\mathbf{x}}_{0|t}, \dots, \bar{\mathbf{x}}_{N|t}\}$ and actions $\bar{\mathbf{U}}_t = \{\bar{\mathbf{u}}_{0|t}, \dots, \bar{\mathbf{u}}_{N-1|t}\}$ that ensure constraint compliance regardless of the realization of $\mathbf{w}_t \in \mathbb{W}_{\mathcal{T}}$. This is achieved by solving the following quadratic program (QP) (e.g., via the solver [49]):

$$\begin{aligned} \bar{\mathbf{U}}_t^*, \bar{\mathbf{X}}_t^* = \underset{\bar{\mathbf{U}}_t, \bar{\mathbf{X}}_t}{\text{argmin}} & \|\mathbf{e}_{N|t}\|_{\mathbf{P}_x}^2 + \sum_{i=0}^{N-1} \|\mathbf{e}_{i|t}\|_{\mathbf{Q}_x}^2 + \|\mathbf{u}_{i|t}\|_{\mathbf{R}_u}^2 \\ \text{subject to } & \bar{\mathbf{x}}_{i+1|t} = \mathbf{A}\bar{\mathbf{x}}_{i|t} + \mathbf{B}\bar{\mathbf{u}}_{i|t}, \\ & \bar{\mathbf{x}}_{i|t} \in \mathbb{X} \ominus \mathbb{Z}, \quad \bar{\mathbf{u}}_{i|t} \in \mathbb{U} \ominus \mathbf{K}\mathbb{Z}, \\ & \mathbf{x}_t \in \mathbb{Z} \oplus \bar{\mathbf{x}}_{0|t}, \quad i = 0, \dots, N-1 \end{aligned} \quad (10)$$

where $\mathbf{e}_{i|t} = \bar{\mathbf{x}}_{i|t} - \mathbf{x}_{i|t}^{\text{des}}$ is the tracking error. The matrix $\mathbf{R}_u$ (positive definite) and $\mathbf{Q}_x$ (positive semi-definite) define the trade-off between deviations from the desired trajectory and actuation usage, while $\|\mathbf{e}_{N|t}\|_{\mathbf{P}_x}^2$ is the terminal cost. $\mathbf{P}_x$ and $\mathbf{K}$ are obtained by formulating an infinite horizon optimal control LQR problem using $\mathbf{A}$, $\mathbf{B}$, $\mathbf{Q}_x$ and $\mathbf{R}_u$ and by solving the associated algebraic Riccati equation [50]. To achieve recursive feasibility, we ensure a sufficiently long prediction horizon is selected, as commonly practiced [51], while omitting the inclusion of terminal set constraints.

Tube and Ancillary Controller. A control input for the real system is generated by RTMPC via an *ancillary controller*:

$$\mathbf{u}_t = \bar{\mathbf{u}}_t^* + \mathbf{K}(\mathbf{x}_t - \bar{\mathbf{x}}_t^*), \quad (11)$$

where $\bar{\mathbf{u}}_t^* = \bar{\mathbf{u}}_{0|t}^*$ and $\bar{\mathbf{x}}_t^* = \bar{\mathbf{x}}_{0|t}^*$. As shown in Fig. 3, This controller ensures that the system remains inside a *tube* (with “cross-section” $\mathbb{Z}$) centered around $\bar{\mathbf{x}}_t^*$ regardless of the realization of the disturbances in $\mathbb{W}_{\mathcal{T}}$, provided that the tube contains the initial state of the system (constraint $\mathbf{x}_t \in \mathbb{Z} \oplus \bar{\mathbf{x}}_{0|t}$). The set $\mathbb{Z}$ is a disturbance invariant set for the closed-loop system $\mathbf{A}_K := \mathbf{A} + \mathbf{B}\mathbf{K}$, satisfying the property that $\forall \mathbf{x}_j \in \mathbb{Z}$, $\forall \mathbf{w}_j \in \mathbb{W}_{\mathcal{T}}$, $\forall j \in \mathbb{N}^+$, $\mathbf{x}_{j+1} = \mathbf{A}_K \mathbf{x}_j + \mathbf{w}_j \in \mathbb{Z}$ [35]. $\mathbb{Z}$ can be computed offline using $\mathbf{A}_K$ and the model of the disturbance $\mathbb{W}$ via ad-hoc analytic algorithms [1], [35], or can be learned from data [52]. Note that tracking aggressive trajectories may introduce large deviations from the operating points, resulting in linearization errors; these errors are treated as an additional source of process uncertainty when computing the tube. In addition, aggressive changes of the reference may result in infeasibility (e.g., when the terminal region is unreachable within the horizon, see [53]), which can be addressed, as

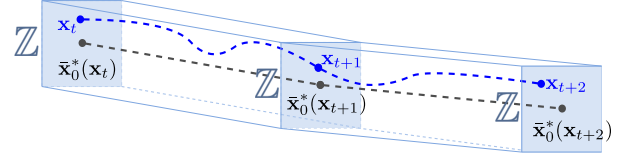


Fig. 3: Illustration of the sequence of robust control invariant sets $\mathbb{Z} \oplus \bar{\mathbf{x}}_0^*(\mathbf{x}_t)$ computed by RTMPC for a system with state $\mathbf{x}_t$ and dimension $n_x = 2$.

typical in MPC, via an adequate choice of the planning horizon ($N = 20$ or $N = 30$ in our work).

B. Shift Compensation via Sampling Augmentation

Training a policy by collecting demonstrations in a controlled source domain $\mathcal{S}$, with the objective of deploying it in a perturbed target domain $\mathcal{T}$ introduces a sample selection bias [54], i.e., data is not collected around the distribution encountered in $\mathcal{T}$. Such bias is a known cause of distribution shifts [54], and can be mitigated by re-weighting collected samples based on their likelihood of appearing in the target domain $\mathcal{T}$ via importance-sampling [23]. Importance-sampling, however, does not apply in our case, since we do not have access to samples/demonstrations collected in $\mathcal{T}$.

In this work, distribution shifts are addressed by additionally utilizing the tube in RTMPC to obtain knowledge of the states that the system may visit when subjected to perturbations in $\mathcal{T}$. Given this information, we propose a tube-guided DA strategy, called Sampling Augmentation (SA), that samples states from the tube and *efficiently* computes corresponding actions via the ancillary controller in RTMPC.

Tube as a Model of State Distribution Under Uncertainties. The key intuition of the proposed approach is the following. We observe that, although the density function of $p(\xi|\pi_{\theta}, \mathcal{T})$ is unknown, an approximation of its support $\mathfrak{R}$, given a demonstration ξ collected in the source domain $\mathcal{S}$, is known and corresponds to the tube in RTMPC when collecting ξ :

$$\mathfrak{R}_{\xi^+|\pi_{\theta^*}, \xi} = \{\bar{\mathbf{x}}_t^* \oplus \mathbb{Z}\}_{t=0}^{T-1}. \quad (12)$$

where ξ^+ is a trajectory in the tube of ξ . This is true thanks to the ancillary controller in Eq. (11), which ensures that the system remains inside Eq. (12) for every possible realization of $\mathbf{w} \in \mathbb{W}_{\mathcal{T}}$. The ancillary controller additionally provides a *computationally efficient* way to obtain the actions to apply for every state inside the tube. Let $\mathbf{x}_{t,j}^+ \in \bar{\mathbf{x}}_t^* \oplus \mathbb{Z}$, i.e., $\mathbf{x}_{t,j}^+$ is a state inside the tube computed when the system is at $\mathbf{x}_t$, then the corresponding robust control action $\mathbf{u}_{t,j}^+$ is:

$$\mathbf{u}_{t,j}^+ = \bar{\mathbf{u}}_t^* + \mathbf{K}(\mathbf{x}_{t,j}^+ - \bar{\mathbf{x}}_t^*). \quad (13)$$

For every timestep t in ξ , extra state-action samples $(\mathbf{x}_{t,j}^+, \mathbf{u}_{t,j}^+)$, with $j = 1, \dots, N_s$ collected from within the tube can be used to augment the dataset employed to train the policy, obtaining a way to approximate the expected risk in the domain $\mathcal{T}$ by only having access to demonstrations collected in $\mathcal{S}$:

$$\begin{aligned} \mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{T})} \mathcal{L}(\theta, \theta^*|\xi) &\approx \\ \mathbb{E}_{p(\xi|\pi_{\theta}, \mathcal{S})} [\mathcal{L}(\theta, \theta^*|\xi) + \mathbb{E}_{p(\xi^+|\pi_{\theta^*}, \xi)} \mathcal{L}(\theta, \theta^*|\xi^+)]. \end{aligned} \quad (14)$$

Tube Approximation and Sampling Strategies. In practice, the density $p(\xi^+|\pi_{\theta^*}, \xi)$ may not be available, making it

Input: $\mathbf{A}, \mathbf{B}, \mathbb{X}, \mathbb{U}, \mathbf{Q}_x, \mathbf{R}_u, \mathbb{W}_T, \beta, \mathcal{S}, \mathbf{X}^{\text{des}}$
Output: Trained policy $\pi_{\hat{\theta}_M}$

```

1:  $\pi_{\theta^*}, \mathbf{K}, \hat{\mathbb{Z}} \leftarrow \text{DesignRtMPC}(\mathbf{A}, \mathbf{B}, \mathbb{X}, \mathbb{U}, \mathbf{Q}_x, \mathbf{R}_u, \mathbb{W}_T)$ 
2:  $\mathcal{D}, \pi_{\hat{\theta}_0} \leftarrow \emptyset, \text{InitializePolicy}()$ 
3: for  $i = 1$  to  $M$  do
4:    $\mathcal{D} \leftarrow \emptyset$  // optional
5:   for  $t = 0$  to  $T - 1$  do
6:      $\mathbf{u}_t^{\text{RTMPC}}, \bar{\mathbf{x}}_t^*, \bar{\mathbf{u}}_t^* \leftarrow \pi_{\theta^*}(\mathbf{x}_t, \mathbf{X}_t^{\text{des}})$  // Eq. (10) and Eq. (11)
7:      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{x}_t, \mathbf{X}_t^{\text{des}}, \mathbf{u}_t^{\text{RTMPC}})\}$ 
8:     for  $j = 1$  to  $N_s$  do
9:        $\mathbf{u}_{t,j}^+ = \bar{\mathbf{u}}_t^* + \mathbf{K}(\mathbf{x}_{t,j}^+ - \bar{\mathbf{x}}_t^*), \mathbf{x}_{t,j}^+ \in \bar{\mathbf{x}}_t^* \oplus \hat{\mathbb{Z}}$ 
10:       $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{x}_{t,j}^+, \mathbf{X}_t^{\text{des}}, \mathbf{u}_{t,j}^+)\}$ 
11:       $\mathbf{u}_t \leftarrow \beta_i \mathbf{u}_t^{\text{RTMPC}} + (1 - \beta_i) \pi_{\hat{\theta}_{i-1}}(\mathbf{x}_t, \mathbf{X}_t^{\text{des}})$  // DAgger/BC
12:       $\mathbf{x}_{t+1} \leftarrow \text{StepSystem}(\mathbf{u}_t, \mathbf{x}_t, \mathcal{S})$  // Sim./Physical Robot
13:       $\pi_{\hat{\theta}_i} \leftarrow \text{UpdatePolicy}(\mathcal{D}, \hat{\theta}_{i-1})$ 

```

Algorithm 1: Sampling Augmentation (SA) for efficient learning from trajectory-tracking linear RTMPC.

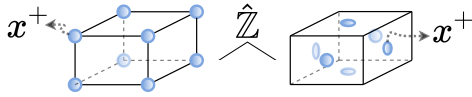


Fig. 4: The possible strategies to sample extra state-action pairs from an axis-aligned bounding box, approximation of robust control invariant set of the RTMPC expert: dense (left) and sparse (right). The diagram is for a system with state dimension $n_x = 3$.

difficult to establish which states to sample for DA. We consider an adversarial approach to the problem by sampling states that may be visited under worst-case perturbations. To efficiently compute those samples, we (outer) approximate the tube $\mathbb{Z}$ with an axis-aligned bounding box $\hat{\mathbb{Z}}$. Note that an axis-aligned bounding box approximation is also used in the design of RTMPC for demonstration collection (Eq. (10)). We investigate two strategies, shown in Fig. 4, to obtain state samples $\mathbf{x}_{t,j}^+$ at every state $\mathbf{x}_t$ in $\mathbb{X}$: i) dense sampling: sample extra states from the vertices of $\bar{\mathbf{x}}_t^* \oplus \hat{\mathbb{Z}}$. The approach produces $N_s = 2^{n_x}$ extra state-action samples. It is more conservative, as it produces more samples, but more computationally expensive. ii) sparse sampling: sample one extra state from the center of each *facet* of $\bar{\mathbf{x}}_t^* \oplus \hat{\mathbb{Z}}$, producing $N_s = 2n_x$ additional state-action pairs. It is less conservative and more computationally efficient.

Algorithm Summary. The procedure is summarized in Algorithm 1. First, SA designs the RTMPC expert according to the uncertainties in the target $\mathbb{W}_T$ (line 1) and randomly initializes the student policy (line 2). Then, SA collects in the source domain $\mathcal{S}$ a demonstration, using DAgger or BC, where β_i is an hyperparameter of DAgger controlling the probability of using actions from the expert and $\beta = 1$ corresponds to BC, storing state and actions in the dataset $\mathcal{D}$ (line 7). The safe plan from the expert is then used to generate extra data via Eq. (13) (line 9), and the policy is updated (line 13, Eq. (4) and Eq. (5) using the data in $\mathcal{D}$ and starting from the previous policy weights $\hat{\theta}_{i-1}$). The data collection and training procedure can be repeated across M demonstrations.

IV. EFFICIENT LEARNING FROM NONLINEAR RTMPC

In this Section, we design an IL and DA strategy, which is an extension of the one presented in Section III, that enables robust and efficient policy learning from an MPC that employs nonlinear models of the form in Eq. (1). Different from Section III, the focus here is on obtaining policies capable of reaching a desired goal state, as this will enable acrobatic

maneuvers – the scenario considered in the evaluation of policies learned from this controller (Section VII). To accomplish this, first, we use a nonlinear version of RTMPC, based on [36], to collect demonstrations that account for the effects of uncertainties. This expert is summarized in Section IV-A. Second, we develop a computationally efficient tube-guided DA strategy leveraging the ancillary controller of the nonlinear RTMPC expert. Unfortunately, unlike in the linear RTMPC case, nonlinear RTMPC [36] uses Nonlinear Model Predictive Control (NMPC) as an ancillary controller. This limits the computational efficiency in DA, as the generation of extra state-action samples requires solving a large NLP associated with the ancillary NMPC (discussed in Section IV-A). We overcome this issue by presenting, in Section IV-B, a time-varying linear feedback law, approximation of the ancillary NMPC, that enables efficient generation of the extra data leveraging the sensitivity of the control input to perturbations in the states visited during an initial demonstration collection procedure. Finally, in Section IV-C, we address the approximation errors introduced by the sensitivity-based DA by presenting strategies to mitigate the gap, in performance and robustness, between the learned policy and the RTMPC expert.

A. Nonlinear RTMPC Expert Formulation

Nonlinear RTMPC [36] ensures state and actuation constraint satisfaction while controlling a nonlinear, uncertain system of the form in Eq. (1). This controller operates by solving two Optimal Control Problems (OCPs), one to compute a nominal safe plan, and one to track the safe plan (ancillary NMPC). **Nominal Safe Planner.** The first OCP, given an $N + 1$ -steps planning horizon, generates nominal safe state and action open-loop plans $\mathbf{Z}_{t_0} = \{\mathbf{z}_{0|t_0}, \dots, \mathbf{z}_{N|t_0}\}$, $\mathbf{V}_{t_0} = \{\mathbf{v}_{0|t_0}, \dots, \mathbf{v}_{N-1|t_0}\}$. The plans are open-loop because they are generated only at time t_0 , when the desired state and action equilibrium pair $\mathbf{X}_{t_0}^{\text{des}} = \{\mathbf{x}_{t_0}^e, \mathbf{u}_{t_0}^e\}$ for the nominal system changes. The nominal safe plan is obtained from:

$$\begin{aligned}
\mathbf{V}_{t_0}^*, \mathbf{Z}_{t_0}^* &= \underset{\mathbf{V}_{t_0}, \mathbf{Z}_{t_0}}{\text{argmin}} J_{\text{RTNMPC}}(\mathbf{Z}_{t_0}, \mathbf{V}_{t_0}, \mathbf{X}_{t_0}^{\text{des}}) \\
\text{subject to } &\mathbf{z}_{i+1|t_0} = f(\mathbf{z}_{i|t_0}, \mathbf{v}_{i|t_0}), \\
&\mathbf{z}_{i|t_0} \in \bar{\mathbb{Z}}, \mathbf{v}_{i|t_0} \in \bar{\mathbb{V}}, \\
&\mathbf{z}_{0|t_0} = \mathbf{x}_{t_0}, \mathbf{z}_{N|t_0} = \mathbf{x}_{t_0}^e.
\end{aligned} \tag{15}$$

$J_{\text{RTNMPC}} = \sum_{i=0}^{N-1} \|\mathbf{z}_{i|t_0} - \mathbf{x}_{t_0}^e\|_{\mathbf{Q}_z}^2 + \|\mathbf{v}_{i|t_0} - \mathbf{u}_{t_0}^e\|_{\mathbf{R}_v}^2$, where $\mathbf{Q}_z, \mathbf{R}_v$ are positive definite. A key idea in this approach involves imposing modified state and actuation constraints $\bar{\mathbb{Z}} \subset \mathbb{X}$ and $\bar{\mathbb{V}} \subset \mathbb{U}$ so that the generated nominal safe plan is at a specific distance from state and actuation constraints. To be more precise, similar to the linear RTMPC case (Eq. (10)), the given state constraints $\mathbb{X}$ and actuation constraints $\mathbb{U}$ are tightened (made more conservative) by an amount that accounts for the spread of trajectories induced by the ancillary controller when the system is subject to uncertainties, obtaining $\bar{\mathbb{Z}} \subset \mathbb{X}$ and $\bar{\mathbb{V}} \subset \mathbb{U}$. Such spread of trajectories corresponds to state and action tubes $\mathbb{T}^{\text{state}} \subset \mathbb{R}^{n_x}, \mathbb{T}^{\text{action}} \subset \mathbb{R}^{n_u}$ that contain the current nominal safe state and action trajectories $\mathbf{z}_{i|t_0}^*, \mathbf{v}_{i|t_0}^*$. Different from the linear case, however, analytically computing the tightened constraints and the tubes is challenging. Fortunately, as highlighted in [36, Section 7], accurately computing these sets is not needed, and an outer approximation is sufficient. This

approximation can be obtained via Monte-Carlo simulations [36] of the system under disturbances, or learned [52]; the procedure employed in our work is tailored to our application domain, and is described in details in Section V-C. Last we note that, as in [36], Eq. (15) is assumed to be feasible.

Ancillary NMPC. The second OCP corresponds to a trajectory tracking NMPC, that acts as an ancillary controller, to maintain the state of the uncertain system close to the reference generated by Eq. (15). The OCP is:

$$\begin{aligned} \bar{\mathbf{u}}_t^*, \bar{\mathbf{x}}_t^* = \underset{\bar{\mathbf{u}}_t, \bar{\mathbf{x}}_t}{\operatorname{argmin}} & \|\mathbf{e}_{N|t}\|_{\mathbf{P}_x}^2 + \sum_{i=0}^{N-1} \|\mathbf{e}_{i|t}\|_{\mathbf{Q}_x}^2 + \|\bar{\mathbf{u}}_{i|t} - \mathbf{v}_{i+t|t_0}^*\|_{\mathbf{R}_u}^2 \\ \text{subject to } & \bar{\mathbf{x}}_{i+1|t} = f(\bar{\mathbf{x}}_{i|t}, \bar{\mathbf{u}}_{i|t}) \\ & \bar{\mathbf{x}}_{0|t} = \mathbf{x}_t, \bar{\mathbf{u}}_{i|t} \in \mathbb{U} \end{aligned} \quad (16)$$

where $\mathbf{e}_{i|t} = \bar{\mathbf{x}}_{i|t} - \mathbf{z}_{i+t-t_0|t_0}^*$ is the state tracking error. The positive definite matrices $\mathbf{Q}_x$ and $\mathbf{R}_u$ are tuning parameters, while $\mathbf{P}_x$ defines a terminal cost. Note that the terminal cost can be set as in [36], or using the solution for the infinite horizon Riccati equation for the linearized system associated with the state at the end of the planning horizon. However, owing to the fact that the expert can use a sufficiently long planning horizon without affecting onboard computation of the learned policy, in our experiments we set the terminal cost to $\mathbf{Q}_x$, additionally demonstrating that our approach introduces opportunities to simplify control design. Eq. (16) is solved at each timestep using the current state $\mathbf{x}_t$, while the action applied to the robot is $\mathbf{u}_t = \bar{\mathbf{u}}_{0|t}^*$. We note that the ancillary NMPC can have different tuning parameters than Eq. (15) providing additional degrees of freedom to shape the response of the system under uncertainties.

A key result (presented in [36, Section 5]) of the employed nonlinear RTMPC [36] is that the ancillary NMPC in Eq. (16) maintains the trajectories of the uncertain system in Eq. (1) inside state and action tubes $\mathbb{T}^{\text{state}}, \mathbb{T}^{\text{action}}$ that contain the current nominal safe state and action trajectories $\mathbf{z}_{t|t_0}^*, \mathbf{v}_{t|t_0}^*$ from the OCP in Eq. (15). The state and action tubes are used to obtain the tightened state and actuation constraints $\bar{\mathbb{Z}}, \bar{\mathbb{V}}$, ensuring constraint satisfaction.

Solving the Ancillary NMPC A large portion of the computational cost of deploying or collecting demonstrations from nonlinear RTMPC comes from the need to solve the OCP of the ancillary NMPC (Eq. (16)) at each timestep. In contrast, the OCP of the nominal safe plan (Eq. (15)) can be solved once per task (e.g., whenever the desired goal state $\mathbf{X}_{t_0}^{\text{des}}$ changes). A state-of-the-art method to solve the optimization in Eq. (16) is sequential quadratic program (SQP), i.e., by repeatedly: i) linearizing the NLP around a given linearization point; ii) generating and solving a corresponding QP, obtaining a refined linearization point for the next SQP iteration. While capable of producing high-quality solutions, SQP methods incur large computational requirements due to computationally-expensive system linearizations, and solving the associated QP one or more times per timestep.

B. Computationally-Efficient Data Augmentation using the Parametric Sensitivities

The tube $\mathbb{T}^{\text{state}}$ induced by the ancillary controller in Eq. (16) identifies relevant regions of the state space for DA, as it

approximates the support of the state distribution under uncertainties, as discussed in Section III-B. However, generating the corresponding extra action samples using Eq. (16) can be very computationally inefficient, as it requires solving the associated SQP for every extra state sample, making DA computationally impractical, and defeating our initial objective of designing *computationally efficient* DA strategies.

In this work, Sampling Augmentation (SA), is extended to efficiently learn policies from nonlinear RTMPC by employing a time-varying, linear approximation of the ancillary NMPC – enabling efficient generation of extra state-action samples. Specifically, we observe that Eq. (16) solves the implicit feedback law:

$$\mathbf{u}_t = \bar{\mathbf{u}}_{0|t}^*(\chi_t) := \kappa(\chi_t), \quad \chi_t := \{\mathbf{x}_t, t; \mathbf{V}_{t_0}^*, \mathbf{Z}_{t_0}^*\} \quad (17)$$

where the current inputs are denoted χ_t . Then, for each timestep of the trajectory collected during a demonstration in the source environment $\mathcal{S}$, with current ancillary NMPC input $\tilde{\chi}_t = \{\tilde{\mathbf{x}}_t, t; \mathbf{V}_{t_0}^*, \mathbf{Z}_{t_0}^*\}$, we generate a local linear approximation of Eq. (17) by computing the first-order sensitivity of $\mathbf{u}_t$ to the initial state $\mathbf{x}_t$:

$$\mathbf{K}_{\tilde{\chi}_t} := \left. \frac{\partial \bar{\mathbf{u}}_{0|t}^*}{\partial \mathbf{x}_t} \right|_{\chi_t = \tilde{\chi}_t} = \left[\left. \frac{\partial \bar{\mathbf{u}}_{0|t}^*}{\partial [\mathbf{x}_t]_1} \right|_{\tilde{\chi}_t}, \dots, \left. \frac{\partial \bar{\mathbf{u}}_{0|t}^*}{\partial [\mathbf{x}_t]_{n_x}} \right|_{\tilde{\chi}_t} \right]. \quad (18)$$

The sensitivity matrix $\mathbf{K}_{\tilde{\chi}_t} \in \mathbb{R}^{n_u \times n_x}$, enables us to compute extra actions $\mathbf{u}_{t,j}^+$ from states inside the tube $\mathbf{x}_{t,j}^+ \in \mathbb{T}^{\text{state}}$, with $j = 1, \dots, N_s$, sampled from the tube:

$$\mathbf{u}_{t,j}^+ = \bar{\mathbf{u}}_{0|t}^* + \mathbf{K}_{\tilde{\chi}_t}(\mathbf{x}_{t,j}^+ - \bar{\mathbf{x}}_{0|t}^*) := \hat{\kappa}(\mathbf{x}_{t,j}^+, \tilde{\chi}_t). \quad (19)$$

The DA procedure enabled by this approximation is computationally-efficient, as we do not need to solve an SQP for each extra state-action sample $(\mathbf{x}_{t,j}^+, \mathbf{u}_{t,j}^+)$ generated for DA, and we only need to compute, once per timestep, the sensitivity matrix $\mathbf{K}_{\tilde{\chi}_t}$. Note that the linearization points of Eq. (18) are based on the trajectory ξ executed during demonstration collection. in the source environment $\mathcal{S}$. We remark, additionally, that the actions computed when collecting demonstrations are obtained by solving the entire SQP, and the sensitivity-based approximation is used only for DA.

Sensitivity Matrix Computation. As described in [2, §8.6], an expression to compute the sensitivity matrix in Eq. (18) (also called *tangential predictor*) can be obtained by re-writing the NLP in Eq. (16) in a parametric form $\mathbf{p}([\mathbf{x}_t]_i)$, highlighting the dependency on scalar parameter representing the i -th component of the initial state $\mathbf{x}_t$ (part of χ_t). The parametric NLP $\mathbf{p}_{\mathcal{X}_t}([\mathbf{x}_t]_i)$ is:

$$\begin{aligned} & \min_{\mathbf{y}} F_{\chi_t}(\mathbf{y}) \\ & \text{subject to } G_{\chi_t}([\mathbf{x}_t]_i, \mathbf{y}) = \mathbf{0} \\ & \quad H(\mathbf{y}) \leq \mathbf{0}, \end{aligned} \quad (20)$$

where $\mathbf{y} \in \mathbb{R}^{n_y}$ corresponds to the optimization variables in Eq. (16), and $F_{\chi_t}(\cdot), G_{\chi_t}(\cdot), H(\cdot)$ are, respectively, the objective function, equality, and inequality constraints in Eq. (16), given the current state and reference trajectory in χ_t . Additionally, we denote the solution of Eq. (20) at $\tilde{\chi}_t$ (computed during the collected demonstration) as $(\tilde{\mathbf{y}}^*, \tilde{\lambda}^*, \tilde{\mu}^*)$, where $\tilde{\lambda}^*, \tilde{\mu}^*$ are, respectively, the Lagrange multipliers for the equality and inequality constraints at the solution found. Then, each i -th

column of the sensitivity matrix (Eq. (18)) can be computed by solving the QP ([2, Th. 8.16]), denoted $\mathbf{p}_{\mathcal{X}_t, L}([\mathbf{x}_t]_i)$:

$$\begin{aligned} \min_{\mathbf{y}} \quad & F_{\mathcal{X}_t, L}(\mathbf{y}; \tilde{\mathbf{y}}^*) + \frac{1}{2}(\mathbf{y} - \tilde{\mathbf{y}}^*)^\top \nabla_{\mathbf{y}}^2 \mathcal{L}(\tilde{\mathbf{y}}^*, \tilde{\boldsymbol{\lambda}}^*, \tilde{\boldsymbol{\mu}}^*)(\mathbf{y} - \tilde{\mathbf{y}}^*) \\ \text{s.t.} \quad & G_{\mathcal{X}_t, L}([\mathbf{x}_t]_i; \mathbf{y}; \tilde{\mathbf{y}}^*) = \mathbf{0} \\ & H_L(\mathbf{y}; \tilde{\mathbf{y}}^*) \leq \mathbf{0} \end{aligned} \quad (21)$$

where $F_{\mathcal{X}_t, L}(\cdot; \tilde{\mathbf{y}}^*)$, $G_{\mathcal{X}_t, L}(\cdot; \tilde{\mathbf{y}}^*)$, $H_L(\cdot; \tilde{\mathbf{y}}^*)$ denote the respective functions in Eq. (20) linearized at the solution found. $\nabla_{\mathbf{y}}^2 \mathcal{L}$ denotes the Hessian of the Lagrangian associated with Eq. (20), while the parameter is perturbed (e.g., $[\mathbf{x}_t]_i \leftarrow [\mathbf{x}_t]_i + 1$). The i -th column of the sensitivity matrix can be extracted from the entries of $\mathbf{y}^*$, solution of Eq. (21), at the position corresponding to $\bar{\mathbf{u}}_{0|t}$. We highlight that Eq. (21) can be computed efficiently, as it leverages the latest internal linearization of the Karush–Kuhn–Tucker (KKT) conditions performed in the SQP employed to solve Eq. (16), and therefore it does not require to re-execute the computationally expensive system linearization routines that are carried out at each SQP iteration. We note that this local approximation exists when the assumptions in [2, Th. 8.15] are satisfied, i.e., that the solution $(\tilde{\mathbf{y}}^*, \tilde{\boldsymbol{\lambda}}^*, \tilde{\boldsymbol{\mu}}^*)$ found during demonstration collection is a strongly regular KKT point, and satisfies strict complementary conditions. Last, extra samples are generated using Eq. (19) under the assumption that the set of active inequality constraints (i.e., the index set $p \in \{1, \dots, n_H\}$ such that $[H(\tilde{\mathbf{y}}^*)]_p = 0$) does not change.

Generalized Tangential Predictor. A strategy that applies to the cases where strict complementary conditions do not hold, or where the extra state samples cause a change in the active set of constraints, is based on the *generalized tangential predictor* [2, §8.9.1]. This predictor can be obtained by solving the QP in Eq. (21) with the set of equality constraints modified to be $G_{\mathcal{X}_t, L}(\mathbf{x}_{t,j}^+, \mathbf{y}; \tilde{\mathbf{y}}) = \mathbf{0}$ [2, Eq. 8.60]. Although this approach requires solving a QP to compute the action $\mathbf{u}_{t,j}^+$ corresponding to each state $\mathbf{x}_{t,j}^+$ sampled from the tube, it does not require re-generating the computationally expensive linearization performed at each SQP iteration (and other performance optimization routines, such as condensing [2]) nor solving the entire SQP for multiple iterations – resulting in a much more computationally-efficient procedure than solving the entire SQP *ex-novo*. We remark that the linearization point in Eq. (21) is updated at every timestep when a full SQP is solved for demonstration-collection.

C. Robustness and Performance Under Approximate Samples

While the described sensitivity-based DA strategy enables the efficient generation of extra state-action samples, it introduces approximation errors that may affect the performance and robustness of the learned policy. Here, we discuss strategies to account for these errors, reducing the gaps between the nonlinear RTMPC expert and the learned policy in terms of robustness and performance.

Robustness. A key property of RTMPC is the ability to explicitly account for uncertainties, including the ones introduced by the proposed sensitivity-based DA framework, by further tightening state and actuation constraints for the nominal safe plan (Eq. (15)). The general nonlinear formulation of the dynamics in Eq. (1), however, makes it challenging to compute an *exact* additional tightening bound for state and

Input: $f(\cdot), \mathbb{X}, \mathbb{U}, \mathbf{Q}_x, \mathbf{R}_u, \mathbb{W}_T, \beta, \mathcal{S}, \mathbf{X}_{t_0}^{\text{des}}$
Output: Trained policy $\pi_{\hat{\boldsymbol{\theta}}_{M+L}}$

```

1:  $\pi_{\boldsymbol{\theta}^*}, \mathbb{T}^{\text{states}} \leftarrow \text{DesingNRTMPC}(f(\cdot), \mathbb{X}, \mathbb{U}, \mathbf{Q}_x, \mathbf{R}_u, \mathbb{W}_T, \mathbf{X}_{t_0}^{\text{des}})$ 
2:  $\mathbf{Z}_{t_0}^*, \mathbf{V}_{t_0}^* \leftarrow \text{GetNominalSafePlan}(\pi_{\boldsymbol{\theta}^*})$  // Eq. (15)
3:  $\kappa \leftarrow \text{GetAncillaryNmPC}(\pi_{\boldsymbol{\theta}^*})$  // Eq. (16), Eq. (17)
4: for  $i = 1$  to  $M$  do
5:   for  $t = 0$  to  $T$  do
6:      $\mathcal{X}_t \leftarrow (\mathbf{x}_t, t; \mathbf{V}_{t_0}^*, \mathbf{Z}_{t_0}^*)$  // Current operating point
7:      $\mathbf{u}_t^{\text{N-RTMPC}}, \mathbf{p}_{\mathcal{X}_t, L} \leftarrow \kappa(\mathcal{X}_t)$  // Eq. (16), save QP Eq. (21)
8:      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{x}_t, \mathbf{X}_{t_0}^{\text{des}}, t, \mathbf{u}_t^{\text{N-RTMPC}})\}$ 
9:      $\mathbf{K}_{\mathcal{X}_t} \leftarrow \text{Sensitivity}(\mathbf{p}_{\mathcal{X}_t, L})$  // Eq. (18)
10:    for  $j = 1$  to  $N_s$  do
11:       $\mathbf{u}_{t,j}^+ = \bar{\mathbf{u}}_t^* + \mathbf{K}_{\mathcal{X}_t}(\mathbf{x}_{t,j}^+ - \bar{\mathbf{x}}_t^*)$ ,  $\mathbf{x}_{t,j}^+ \in \bar{\mathbf{x}}_t^* \oplus \mathbb{T}^{\text{states}}$ 
12:       $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{x}_{t,j}^+, \mathbf{X}_{t_0}^{\text{des}}, t, \mathbf{u}_{t,j}^+)\}$ 
13:       $\mathbf{u}_t \leftarrow \beta_i \mathbf{u}_t^{\text{N-RTMPC}} + (1 - \beta_i) \pi_{\hat{\boldsymbol{\theta}}_{i-1}}(\mathbf{x}_t, \mathbf{X}_{t_0}^{\text{des}})$  // DAgger/BC
14:       $\mathbf{x}_{t+1} \leftarrow \text{StepSystem}(\mathbf{u}_t, \mathbf{x}_t, \mathcal{S})$ 
15:       $\pi_{\hat{\boldsymbol{\theta}}_i} \leftarrow \text{UpdatePolicy}(\mathcal{D}, \hat{\boldsymbol{\theta}}_{i-1})$ 
16:    if FineTuning then
17:       $\mathcal{D} \leftarrow \emptyset$ 
18:      for  $l = 1$  to  $L$  do
19:         $\boldsymbol{\xi} = \{(\mathbf{x}_t, \mathbf{X}_{t_0}^{\text{des}}, \mathbf{u}_t^{\text{N-RTMPC}})\}_{t=0}^{T-1}$ 
20:         $\leftarrow \text{CollectDemo}(\kappa, \mathbf{Z}_{t_0}^*, \mathbf{V}_{t_0}^*, \pi_{\hat{\boldsymbol{\theta}}_{M+l-1}}, \beta_i, \mathcal{S})$  // DAgger/BC
21:         $\mathcal{D} \leftarrow \mathcal{D} \cup \{\boldsymbol{\xi}\}$ 
22:       $\pi_{\hat{\boldsymbol{\theta}}_{M+l}} \leftarrow \text{UpdatePolicy}(\mathcal{D}, \hat{\boldsymbol{\theta}}_{M+l-1})$ 
```

Algorithm 2: Sampling Augmentation for efficient learning from Nonlinear RTMPC

actuation constraints. A possible avenue to establish a tightening procedure for the actuation constraints is to observe that the linear approximation of Eq. (17) introduces an error upper bounded by ([2, Th. 8.16]):

$$\|\kappa(\mathcal{X}_t) - \hat{\kappa}(\mathbf{x}_{t,j}^+, \mathcal{X}_t)\| \leq D \|\mathbf{x}_{t,j}^+ - \bar{\mathbf{x}}_t\|^2 \quad (22)$$

where D may be obtained by considering the Lipschitz constant of the controller (e.g., [24]). However, estimating this constant may be difficult or computationally expensive for large-dimensional systems, as is the case herein. An alternative is to update the tubes as was done in Section IV-A, e.g., by employing Monte-Carlo simulations of the closed-loop system, starting from an initial (possibly conservative) tightening guess and by iteratively adjusting the cross-section (size) of the tube, or by directly learning the tubes from simulations or previous (conservative) real-world deployments [52]. These iterative procedures are particularly appealing in our context, as our efficient policy learning methodologies enable rapid training/updates of the learned policy, and the computational efficiency of the policy enables rapid numerical validations.

Performance Improvements via Fine-Tuning. In the context of learning policies from nonlinear RTMPC, we include in SA an (optional) fine tuning-step. This fine-tuning step consists in training the policy with additional demonstrations, without DA, therefore avoiding introducing further approximate samples, and having discarded the extra data used to train the policy after an initial demonstration. Therefore, tube-guided DA is treated as a methodology to efficiently generate an initial guess of the policy parameters.

Algorithm Summary. The SA procedure for nonlinear RTMPC with the fine-tuning step is summarized in Algorithm 2. It consists of the following:

- 1) Pre-compute the safe plan from the expert (line 2).
- 2) Collect a single ($M = 1$) task demonstration $\boldsymbol{\xi}$ that tracks the safe plan using the ancillary NMPC (line 6-14), while

- additionally storing the variables of the QP in Eq. (21).
- 3) Perform DA using the parametric sensitivity (Section IV-B, line 10-12, shown for the case where no active change of constraints occurs and strict complementary conditions hold, else use Eq. (21)) and train the policy, obtaining the parameters $\hat{\theta}_1$ (line 15).
 - 4) Optional *fine-tuning* step (line 16):
 - i) Discard the collected data so far, including the data generated by the DA (line 17).
 - ii) Collect new demonstrations using DAgger [18] and the pre-trained policy, or BC, line 21, and re-train the pre-trained policy (with parameters $\hat{\theta}_1$) after every newly collected demonstration.

V. APPLICATION TO AGILE FLIGHT

In this Section, we tailor the proposed efficient policy learning strategies to agile flight tasks, as this will be the focus of our numerical and experimental evaluation. First, in Section V-A, we present the nonlinear model of the multirotor used to collect demonstrations in simulation. Then, in Section V-B, we present a RTMPC expert for *trajectory tracking* based on a *linear* multirotor model and that will be used with the IL procedure described in Section III. Because the considered trajectories require the robot to operate around a fixed, pre-defined condition (near hover), a hover-linearized model is suitable for the design of this controller. Last, in Section V-C, we design a nonlinear RTMPC expert capable of performing a 360° flip in *near-minimum time* - a maneuver that demands exploitation of the full *nonlinear* dynamics of the multirotor, and that requires large and careful actuation usage; this controller is used with the learning in Section IV.

A. Nonlinear Multirotor Model

We consider an inertial reference frame W attached to the ground, and a non-inertial frame B attached to the center of mass (CoM) of the robot. The translational and rotational dynamics of the multirotor are:

$${}_W\dot{\mathbf{p}} = {}_W\mathbf{v} \quad (23a)$$

$${}_W\dot{\mathbf{v}} = m^{-1}({}_B\mathbf{R}_W \mathbf{t}_{\text{cmd}} + {}_W\mathbf{f}_{\text{drag}} + {}_W\mathbf{f}_{\text{ext}}) - {}_W\mathbf{g} \quad (23b)$$

$$\dot{\mathbf{q}}_W = \frac{1}{2}\Omega({}_B\boldsymbol{\omega})\mathbf{q}_W \quad (23c)$$

$${}_B\dot{\boldsymbol{\omega}} = \mathbf{I}_{\text{mav}}^{-1}(-{}_B\boldsymbol{\omega} \times \mathbf{I}_{\text{mav}} {}_B\boldsymbol{\omega} + {}_B\boldsymbol{\tau}_{\text{cmd}} + {}_B\boldsymbol{\tau}_{\text{drag}}) \quad (23d)$$

where $\mathbf{p}$, $\mathbf{v}$, $\mathbf{q}$, $\boldsymbol{\omega}$ are, respectively, position, velocity, attitude quaternion and angular velocity of the robot, with the prescript denoting the corresponding reference frame. The attitude quaternion $\mathbf{q} = [q_w, \mathbf{q}_v]^\top$ consists of a scalar part q_w and a vector part $\mathbf{q}_v = [q_x, q_y, q_z]^\top$ and it is unit-normalized; the associated 3×3 rotation matrix is $\mathbf{R} = \mathbf{R}(\mathbf{q})$, while

$$\Omega(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\boldsymbol{\omega}^\top \\ \boldsymbol{\omega} & [\boldsymbol{\omega}]_\times \end{bmatrix}, \quad (24)$$

with $[\boldsymbol{\omega}]_\times$ denoting the 3×3 skew symmetric matrix of $\boldsymbol{\omega}$. m denotes the mass, $\mathbf{I}_{\text{mav}}$ the 3×3 diagonal inertial matrix, and $\mathbf{g} = [0, 0, g]^\top$ the gravity vector. Aerodynamic effects are taken into account via $\mathbf{f}_{\text{drag}} = -c_{D,1}\mathbf{v} - c_{D,2}\|\mathbf{v}\|\mathbf{v}$ and isotropic drag torque $\boldsymbol{\tau} = -c_{D,3}\boldsymbol{\omega}$, capturing the parasitic drag produced by the motion of the robot. The robot is additionally subject to

external force disturbances $\mathbf{f}_{\text{ext}}$, such as the one caused by wind or by an unknown payload. Last, $\mathbf{t}_{\text{cmd}} = [0, 0, t_{\text{cmd}}]^\top$ is the commanded thrust force, and $\boldsymbol{\tau}_{\text{cmd}}$ the commanded torque. These commands can be mapped to the desired thrust $f_{\text{prop},i}$ for the i -th propeller ($i = 1, \dots, n_p$) via a linear mapping (*allocation* matrix) $\mathcal{A}$:

$$\begin{bmatrix} t_{\text{cmd}} \\ \boldsymbol{\tau}_{\text{cmd}} \end{bmatrix} = \mathcal{A} \begin{bmatrix} f_{\text{prop},1} \\ \vdots \\ f_{\text{prop},n_p} \end{bmatrix} = \mathcal{A}\mathbf{f}_{\text{prop}}. \quad (25)$$

The attitude of the quadrotor is controlled via the geometric attitude controller in [55]. This controller generates desired torque commands ${}_B\boldsymbol{\tau}_{\text{cmd}}$ given a desired attitude $\mathbf{R}_{WB}^{\text{des}}$, angular velocity ${}_B\boldsymbol{\omega}^{\text{des}}$ and acceleration ${}_B\dot{\boldsymbol{\omega}}^{\text{des}}$ via [55]:

$$\begin{aligned} {}_B\boldsymbol{\tau}_{\text{cmd}} &= -\mathbf{K}_R\mathbf{e}_R - \mathbf{K}_\omega\mathbf{e}_\omega + {}_B\boldsymbol{\omega} \times \mathbf{J}_B\boldsymbol{\omega} \\ &\quad - \mathbf{J}({}_B\boldsymbol{\omega}^\wedge \mathbf{R}_{WB}^\top \mathbf{R}_{WB}^{\text{des}} {}_B\boldsymbol{\omega}^{\text{des}} - \mathbf{R}_{WB}^\top \mathbf{R}_{WB}^{\text{des}} {}_B\dot{\boldsymbol{\omega}}^{\text{des}}), \\ \mathbf{e}_R &= \frac{1}{2}(\mathbf{R}_{WB}^{\text{des}\top} \mathbf{R}_{WB} - \mathbf{R}_{WB}^\top \mathbf{R}_{WB}^{\text{des}})^\vee, \\ \mathbf{e}_\omega &= {}_B\boldsymbol{\omega} - \mathbf{R}_{WB}^\top \mathbf{R}_{WB}^{\text{des}} {}_B\dot{\boldsymbol{\omega}}^{\text{des}}. \end{aligned} \quad (26)$$

The diagonal matrices $\mathbf{K}_R, \mathbf{K}_\omega$ of size 3×3 are tuning parameters of the controller, while $\mathbf{e}_R$ denotes the attitude error, and $\mathbf{e}_\omega$ is its time derivative. The symbol $(\mathbf{r}^\wedge)^\vee = \mathbf{r}$ denotes the operation transforming a 3×3 skew-symmetric matrix $\mathbf{r}^\wedge$ in a vector $\mathbf{r} \in \mathbb{R}^3$. The position controllers designed in the next sections output setpoints for the attitude controller, and desired thrust t_{cmd} .

B. Linear RTMPC for Trajectory Tracking

The model employed by the linear RTMPC for trajectory tracking (Eq. (10)) is based on a simplified, hover-linearized model derived from Eq. (26), using the approach in [6], but modified to account for uncertainties. First, similar to [6], we express the model in a yaw-fixed, gravity-aligned frame I via the rotation matrix $\mathbf{R}_{BI}$

$$\begin{bmatrix} \phi \\ \theta \end{bmatrix} = \mathbf{R}_{BI} \begin{bmatrix} I\phi \\ I\theta \end{bmatrix}, \quad \mathbf{R}_{BI} = \begin{bmatrix} \cos(\psi) & \sin(\psi) \\ -\sin(\psi) & \cos(\psi) \end{bmatrix}, \quad (27)$$

where the attitude has been represented, for interpretability, via the Euler angles yaw ψ , pitch θ , roll ϕ (*intrinsic* rotations around the z - y - x such that $\mathbf{R} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$, with $\mathbf{R}_l(\alpha)$ being a rotation of α around the l -th axis). Second, as in [6], we assume that the closed-loop attitude dynamics can be described by a first-order dynamical system that can be identified from experiments, replacing Eq. (23c), Eq. (23d). Last, different from [6], we assume ${}_W\mathbf{f}_{\text{ext}}$ in Eq. (23b) to be an unknown disturbance/model errors that capture the uncertain parts of the model, such that ${}_W\mathbf{f}_{\text{ext}} \in \mathbb{W}$.

The controller generates tilt (roll, pitch) and thrust commands ($n_u = 3$) given the state of the robot ($n_x = 8$, consisting of position, velocity, and tilt), and given the reference trajectory. The desired yaw is fixed, and it is tracked by the cascaded attitude controller; similarly, ${}_B\boldsymbol{\omega}^{\text{des}}$ and ${}_B\dot{\boldsymbol{\omega}}^{\text{des}}$ are set to zero. We employ the nonlinear attitude compensation scheme in [6].

The controller takes into account position constraints (e.g., available 3D flight space), actuation limits, and velocity/tilt limits via $\mathbb{X}$ and $\mathbb{U}$. The cross-section of the tube $\mathbb{Z}$ is a constant outer approximation based on an axis-aligned bounding box. It is estimated via Monte-Carlo sampling, by measuring the state deviations of the closed loop linear system $\mathbf{A}_K$ under the disturbances in $\mathbb{W}$.

C. Nonlinear RTMPC for Acrobatic Maneuvers

Ancillary NMPC. We start by designing the ancillary NMPC (Eq. (16)). The selected nominal model is the same used in the high-performance trajectory tracking NMPC for multirotors [22]:

$$\begin{aligned} {}_W\dot{\mathbf{p}} &= {}_W\mathbf{v} \\ {}_W\dot{\mathbf{v}} &= m^{-1}(\mathbf{R}_{WB}\mathbf{t}_{\text{cmd}} + {}_W\mathbf{f}_{\text{drag}}) - {}_W\mathbf{g} \\ \dot{\mathbf{q}}_{WB} &= \frac{1}{2}\boldsymbol{\Omega}(\mathbf{B}\boldsymbol{\omega}_{\text{cmd}})\mathbf{q}_{WB}, \end{aligned} \quad (28)$$

where the rotational dynamics (Eq. (23d)) have been neglected, assuming that the cascaded attitude controller enables fast tracking of the desired angular velocity setpoint ${}_B\boldsymbol{\omega}_{\text{cmd}}$. The controller uses the state and control input:

$$\bar{\mathbf{x}} = [{}_W\mathbf{p}^\top, {}_W\mathbf{v}^\top, \mathbf{q}_{WB}^\top]^\top, \quad \bar{\mathbf{u}} = [t_{\text{cmd}}, {}_B\boldsymbol{\omega}_{\text{cmd}}^\top]^\top. \quad (29)$$

The feed-forward angular acceleration for the attitude controller ${}_B\dot{\boldsymbol{\omega}}_{\text{cmd}}$ is obtained via numerical differentiation. We do not explicitly generate an attitude setpoint (we set $\mathbf{R}_{WB}^{\text{des}} = \mathbf{R}_{WB}$), so that Eq. (28) acts as a proportional body-rates controller with feed-forward accelerations.

Near-Minimum Time Safe Plan Generation. To compute safe nominal plans for acrobatic maneuvers (by solving the OCP in Eq. (15)), we employ an extended version of the full nonlinear dynamic model in Section V-A. More specifically, we solve the OCP in Eq. (15) by using the following state $\tilde{\mathbf{z}} \in \tilde{\mathbb{Z}}$ and control inputs $\tilde{\mathbf{v}} \in \tilde{\mathbb{V}}$:

$$\tilde{\mathbf{z}} = [{}_W\mathbf{p}^\top, {}_W\mathbf{v}^\top, \mathbf{q}_{WB}^\top, {}_B\boldsymbol{\omega}^\top, {}_B\mathbf{f}_{\text{prop}}^\top]^\top, \quad \tilde{\mathbf{v}} = {}_B\dot{\mathbf{f}}_{\text{prop}}, \quad (30)$$

where the state has been extended to include the thrust produced by each propeller $\mathbf{f}_{\text{prop}}$ to ensure continuity in the reference thrust, accounting for the unmodeled actuators' dynamics. As for the linear case, uncertainties are modeled by ${}_W\mathbf{f}_{\text{ext}} \in \mathbb{W}$. The cost function captures the near-minimum time objective:

$$\tilde{J}_{\text{RTNMPC}} = T_f + \alpha_1 \mathbf{v}^\top \mathbf{v} + \alpha_2 \mathbf{f}_{\text{prop}}^\top \mathbf{f}_{\text{prop}} + \alpha_3 \tilde{\mathbf{v}}^\top \tilde{\mathbf{v}} \quad (31)$$

where T_f is the total time of the maneuver, while the remaining terms act as a regularizer for the optimizer, with $\alpha_i \ll T_f$ (i.e., $\alpha_i \approx 10^{-2}$, $\forall i$).

We note that $\tilde{J}_{\text{RTNMPC}}$ contains a non-quadratic term, therefore differing from the quadratic cost employed in the safe nominal planner in [36] (our Eq. (15)); such cost function was chosen to automate the selection of the prediction horizon N for the safe nominal plan. Our evaluation will demonstrate that the ancillary NMPC maintains the system within a tube from the generated reference, further highlighting the flexibility of the framework.

Additionally, we note that state and control input (Eq. (30)) have been extended compared to the ones (Eq. (29)) selected for the ancillary NMPC, as emphasized by our notation $\tilde{\cdot}$. For this reason, the optimal safe nominal plan $\tilde{\mathbf{z}}_t^*$, $\tilde{\mathbf{v}}_t^*$ found using the extended state needs to be mapped to the reference trajectory for the ancillary NMPC, $\mathbf{z}_t^*$, $\mathbf{v}_t^*$. This is done by simply selecting position, velocity and attitude from $\tilde{\mathbf{z}}_t^*$ to obtain $\mathbf{z}_t^*$. The thrust setpoint t_{cmd} in $\mathbf{v}_t^*$ is computed via $\mathcal{A}$ in Eq. (25) from $\mathbf{f}_{\text{prop}}$ in $\tilde{\mathbf{z}}_t^*$, while the angular velocity setpoint $\boldsymbol{\omega}_{\text{cmd}}$ is obtained by assuming it equal to the angular velocity $\boldsymbol{\omega}$ in $\tilde{\mathbf{z}}_t^*$.

Constraints. The state constraint $\bar{\mathbf{x}}_t \in \mathbb{X}$ encodes the maximum safe linear velocity $\mathbf{v}$ and position boundaries $\mathbf{p}$ of the

environment, while actuation constraints $\bar{\mathbf{u}}_t \in \mathbb{U}$ account for physical limits of the robot, restricting the nominal angular velocities $\boldsymbol{\omega}_{\text{cmd}}$ (to prevent saturation of the onboard gyroscope), and the maximum/minimum thrust force t_{cmd} produced by the propellers. We impose tightened constraints on the thrust force by constraining $\mathbf{f}_{\text{prop}}$ in $\tilde{\mathbf{z}} \in \tilde{\mathbb{Z}}$. These constraints are obtained via a conservative approach, i.e. we require a minimal thrust to generate a trajectory feasible within our position and velocity constraints. Such feasible trajectory is found via an iterative tightening procedure for the thrust constraints, using the previously-obtained feasible trajectory as an initial guess for the subsequent optimization under tightened thrust constraints. This procedure ensures that sufficient control authority is left to the ancillary NMPC to account for the presence of large unknown aerodynamic effects and mismatches in the mapping from commanded thrust/actual thrust. This cautious approach enabled successful real-world execution of the maneuver without further real-world tuning. We additionally leverage the further degrees of freedom introduced by the extended state $\tilde{\mathbf{z}}$ by shaping the safe plan through upper-bounding the thrust rates $\dot{\mathbf{f}}_{\text{prop}}$ via $\tilde{\mathbf{v}}$, although this constraint will not be enforced by the ancillary NMPC. Last, using Monte-Carlo closed-loop simulations with disturbances sampled from $\mathbb{W}$, we verify that $\mathbb{X}$ and $\mathbb{U}$ are satisfied, and we generate a constant estimate (outer approximation, axis-aligned bounding box) of the cross-section of the tubes $\mathbb{T}^{\text{state}}$ and $\mathbb{T}^{\text{action}}$.

Tube and Data Augmentation with Attitude Quaternions.

The normalized attitude quaternion, part of the states $\bar{\mathbf{x}}$, $\tilde{\mathbf{z}}$ of nonlinear RTMPC, and part of the reference $\mathbf{z}_t^*$ for the ancillary NMPC, does not belong to a vector space, and therefore it is not trivial to describe its tube nor to generate extra samples for DA. In this work, we employ an attitude error representation $\boldsymbol{\epsilon} \in \mathbb{R}^3$ based on the Modified Rodriguez Parameters (MRP) [56] to generate a representation that can be treated as an element of a vector space. Specifically, we use

$$\boldsymbol{\epsilon}_t = \text{MRP}(\mathbf{q}_t \odot \mathbf{q}_t^{*-1}), \quad (32)$$

where $\mathbf{q}_t$ is the current attitude, $\mathbf{q}_t^*$ is the desired attitude (from the safe plan $\mathbf{z}_t^*$), $\text{MRP}(\cdot)$ maps a quaternion to the corresponding three-dimensional attitude representation, while $\odot$ denotes the quaternion product.

VI. EVALUATION - LEARNING FROM LINEAR RTMPC

We start by evaluating our policy learning approach for the task of trajectory tracking using the linear RTMPC expert.

A. Evaluation Approach and Details

Simulation Environment. Demonstration collection and policy evaluations are performed in a simulation environment implementing the nonlinear multirotor dynamics in Section V-A, discretized at 400 Hz, while the attitude controller runs at 200 Hz. The robot follows desired trajectories, starting from randomly generated initial states centered around the origin. Given the specified external disturbance magnitude bound $\mathbb{W}_{\mathcal{E}} = \{\mathbf{f}_{\text{ext}} \in \mathbb{R}^3 | \underline{\mathbf{f}}_{\text{ext}} \leq \mathbf{f}_{\text{ext}} \leq \bar{\mathbf{f}}_{\text{ext}}\}$, disturbances are applied in the domain $\mathcal{E}$ by sampling ${}_W\mathbf{f}_{\text{ext}}$ via the spherical coordinates:

$${}_W\mathbf{f}_{\text{ext}} = f_{\text{ext}} \begin{bmatrix} \cos(\phi) \sin(\theta) \\ \sin(\phi) \sin(\theta) \\ \cos(\theta) \end{bmatrix}, \quad \begin{aligned} f_{\text{ext}} &\sim \mathcal{U}(\underline{f}_{\text{ext}}, \bar{f}_{\text{ext}}), \\ \theta &\sim \mathcal{U}(0, \pi), \\ \phi &\sim \mathcal{U}(0, 2\pi). \end{aligned} \quad (33)$$

Linear RTMPC. The linear RTMPC expert demonstrator runs in simulation at 10 Hz, and its tube is designed assuming $\mathbb{W} = \{f_{\text{ext}} \in \mathbb{R} | 0 \leq f_{\text{ext}} \leq 0.35mg\}$, corresponding to the safe physical limit of the actuators of the robot. The reference fed to the expert is a sequence of desired positions and velocities for the next 3s, discretized with a sampling time of 0.1 s; the expert uses a corresponding planning horizon of $N = 30$, (resulting in a reference being a 180-dim. vector).

Policy. The student policy is a 2-hidden layer, fully connected DNN, with (32, 32) or (64, 32) neurons/layer, and ReLU activation function. The input/output size match the ones of the optimization problem solved by the expert in Section V-B: the input has size 188 (state and reference trajectory), while the output has size 3 (thrust, and tilt in an inertial frame).

Baselines and Training Details. We apply the proposed SA strategies to every demonstration collected via DAgger or BC, and we consider DAgger or BC without any augmentation/robustification approach (denoted **n.a.**), or combined with:

- DA (linear interpolation):** a DA that groups the collected demonstrations based on the input reference trajectory (or reference position/time for the go-to-goal-position case), and then randomly samples pairs of input-outputs in each cluster, linearly interpolating the state/action to obtain a new state-action pair.
- DA (expert neighborhood):** a DA strategy that uniformly samples states from a region corresponding to 5% of the cross-section of the tube in RTMPC, centered around the current state of the robot. The corresponding actions are obtained using the ancillary controller. This baseline is useful at studying the importance of using the entire tube as a support of the sampling distribution.
- DR:** domain randomization.

During demonstration-collection in the source environment $\mathcal{S}$, we do not apply disturbances, setting $\mathbb{W}_{\mathcal{S}} = \{\emptyset\}$, with the exception for DR, where we sample disturbances from $\mathbb{W}_{\text{DR}} = \mathbb{W}_{\mathcal{T}}$. In all the methods that use DAgger we set the probability of using actions of the expert β (a hyperparameter of DAgger [18]) to be 1 at the first demonstration and 0 otherwise (as this was found to be the best-performing setup). The number of samples generated for the baseline DA methods is 16 per timesteps, matching the number used for SA-sparse, while SA-dense corresponds to 256 samples per timestep. Demonstrations are collected at control rate (0.1 s). After every collected demonstration, the policy is trained for up to 50 epochs using all the data available so far with the ADAM [57] optimizer and a learning rate of 0.001, and we use early stopping, terminating the training if the validation loss (from 30% of the collected data) does not decrease within 7 epochs. The policy is then evaluated on the task for 10 times (episodes), starting from slightly different initial states centered around the origin, in both $\mathcal{S}$ and $\mathcal{T}$.

Evaluation Metrics: We monitor:

- Robustness (Success Rate),** as the percentage of episodes where the robot never violates any state constraint;
- Performance,** via either
 - $C_{\xi}(\pi_{\theta}) := \sum_{t=0}^T \|\mathbf{x}_t - \mathbf{x}_t^{\text{des}}\|_{\mathbf{Q}_x}^2 + \|\mathbf{u}_t\|_{\mathbf{R}_u}^2$ tracking error along the trajectory (*MPC Stage Cost*); or
 - $\|C_{\xi}(\pi_{\theta^*}) - C_{\xi}(\pi_{\hat{\theta}^*})\| / \|C_{\xi}(\pi_{\theta^*})\|$ relative error between expert and policy tracking errors (*Expert Gap*);

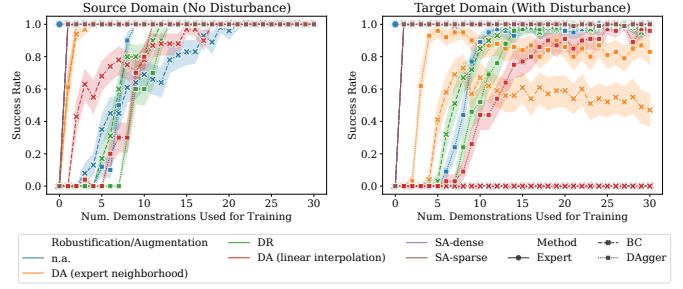


Fig. 5: Robustness (*Success Rate*) in the task of flying along a figure-8 trajectory (7 s long), with wind-like disturbances (right, target domain $\mathcal{T}_1$) and without (left, source domain $\mathcal{S}$), starting from different initial states. Evaluation across 10 random seeds, 10 times per demonstration per seed. Shaded lines are the 95% confidence interval. The lines for the SA-based methods overlap.

TABLE II: Comparison of IL methods to learn a policy from RTMPC. The proposed SA-methods simultaneously achieve high robustness, demonstration efficiency and performance close to the one of the expert, unlike the considered baselines. Note: Robustness and performance are evaluated at convergence (demonstration 20-30 for non-SA methods, and 1-11 for SA-methods). *Demonstration-Efficiency*: number of demonstrations to achieve for the first time an average 100% success rate. *Easy*: no disturbances applied during data collection. *Safe*: no state constrain violations recorded during data collection. *Safe in our numerical evaluation, but not guaranteed as it requires executing actions of a policy that may be partially trained. Color-coding: better: green/white; worse: red.

Method Robustification/ Augmentation	Imitation	Data Collection		Robustness succ. rate (%)		Performance expert gap (%)		Demonstration Efficiency	
		Easy	Safe	$\mathcal{T}_1$	$\mathcal{T}_2$	$\mathcal{T}_1$	$\mathcal{T}_2$	$\mathcal{T}_1$	$\mathcal{T}_2$
n.a.	BC	Yes	Yes	0.0	100.0	22.8	37.2	-	18
	DAgger	Yes	No	97.6	100.0	13.6	2.9	-	9
DA (linear interpolation)	BC	Yes	No	0.0	99.9	23.3	36.7	-	16
	DAgger	Yes	No	92.9	100.0	26.9	3.7	-	12
DA (expert neighborhood)	BC	Yes	Yes	53.5	100.0	27.5	2.7	-	3
	DAgger	Yes	No	83.3	100.0	22.3	2.7	-	2
DR	BC	No	Yes	98.7	100.0	6.8	8.5	15	14
	DAgger	No	No	99.1	100.0	6.7	2.8	20	9
SA-Dense	BC	Yes	Yes	100.0	100.0	6.3	2.8	1	1
	DAgger	Yes	Yes*	100.0	100.0	6.3	2.8	1	1
SA-Sparse	BC	Yes	Yes	99.9	100.0	6.2	2.8	1	1
	DAgger	Yes	Yes*	100.0	100.0	6.3	2.8	1	1

iii) Efficiency

- number of expert demonstrations (*Num. Demonstrations Used for Training*), and
- wall-clock time to generate the policy (*Training Time*²).

B. Numerical Evaluation of Efficiency, Robustness, and Performance when Learning to Track a Single Trajectory

Tasks Description. Our objective is to generate a policy from linear RTMPC capable of tracking a 7s long (70 steps), figure eight-shaped trajectory. We evaluate the considered IL approaches in two different target domains, with wind-like disturbances ($\mathcal{T}_1$) or with model errors ($\mathcal{T}_2$). Disturbances in $\mathcal{T}_1$ are external force perturbations $\mathbf{f}_{\text{ext}}$ sampled from $\mathbb{W}_{\mathcal{T}_1} \approx \{f_{\text{ext}} | 0.25mg \leq f_{\text{ext}} \leq 0.3mg\}$. Model errors in $\mathcal{T}_2$ are applied via mismatches in the drag coefficients used between training and testing, representing uncertainties not explicitly considered during the design of the linear RTMPC.

Comparison with IL baselines. We start by evaluating the robustness in $\mathcal{T}_1$ as a function of the number of demonstrations collected in the source domain. The results are shown in Fig. 5, highlighting that: i) while all the approaches achieve robustness (full success rate) in the source domain, SA achieves full success rate after only a single demonstration, being 3 times

²Training time is the time to collect demonstrations and the time to train the policy, as measured by a wall-clock. In our evaluations, the simulated environment steps at its highest possible rate (in contrary to running at the same rate of the simulated physical system), providing an advantage to those methods that require a large number of environment interactions, such as the considered baselines.

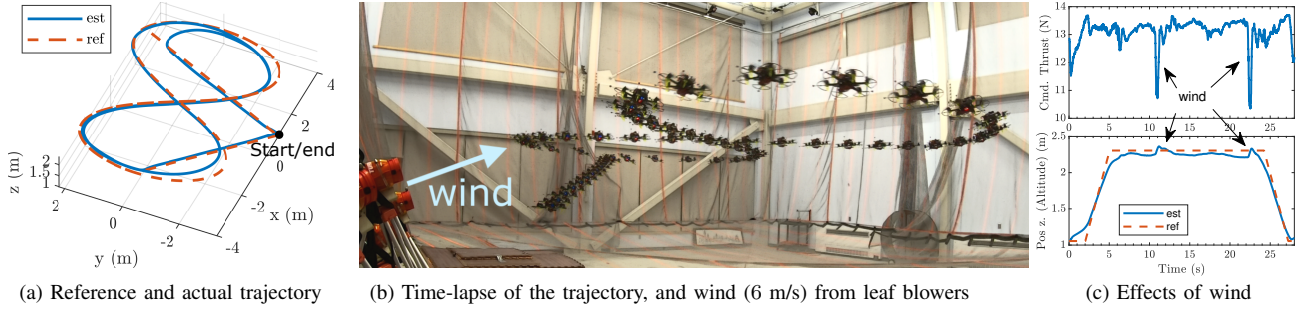


Fig. 6: Experimental evaluation of a trajectory tracking policy learned from a *single* linear RTMPC demonstration collected in simulation, achieving *zero-shot* transfer. The multirotor is able to withstand previously unseen disturbances, such as the wind produced by an array of leaf-blowers, and whose effects are clearly visible in the altitude errors (and change in commanded thrust) in Fig. 6c. This demonstration-efficiency and robustness is enabled by Sampling-Augmentation (SA), our proposed tube-guided data augmentation strategy.

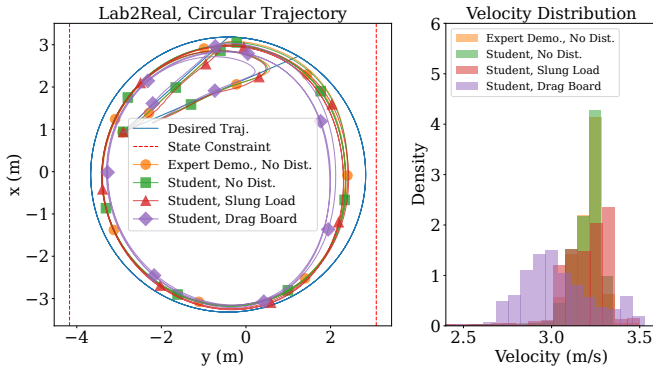


Fig. 7: Example of *lab2real* transfer, where one RTMPC demonstration (Expert Demo.) collected with the actual robot is used to train a policy (Student) that is robust to previously unseen disturbances. Policy runs onboard at 500 Hz.

more sample efficient than the most demonstration-efficient baseline, DA (expert neighborhood), which however does not achieve full robustness in the target domain; ii) SA, instead, is also able to achieve full robustness in the target domain, while baseline methods do not fully succeed or converge at a much lower rate. These results emphasize the presence of a distribution shift between the source and target, which is not fully compensated for by baseline methods such as BC due to a lack of exploration and robustness.

The performance evaluation and additional results are summarized in Table II. We highlight that in the target domain $\mathcal{T}_1$, SA achieves the performance that is closest to the expert. Table II additionally presents the results for the target domain $\mathcal{T}_2$. Although this task is less challenging (i.e., all the approaches achieve full robustness), the proposed method (SA-sparse) achieves the highest demonstration-efficiency and among the lowest expert gap, with similar trends as in $\mathcal{T}_1$.

Training Time. Fig. 5 highlights that the best-performing baseline, DAGger+DR, requires about 10 demonstrations to learn to robustly track a 7s long trajectory, which corresponds to a total training time of 10.8s. Among the proposed approaches, DAGger+SA-sparse instead only requires 1 demonstration, corresponding to a training time of 3.8s, a 64.8% reduction in wall-clock time required to learn the policy. DAGger+SA-dense, instead, while requiring a single demonstration to achieve full robustness, necessitates 114s of training time due to the large number of samples generated. Due to its effectiveness and greater computational efficiency, we use SA-sparse rather than SA-dense for the rest of the evaluations.

C. Hardware Evaluation for Tracking a Single Trajectory from a Single Demonstration

Sim2Real Transfers. We validate the demonstration-efficiency, robustness, and performance of the proposed approach by experimentally testing policies trained after a *single* demonstration collected in simulation using DAGger/BC (which operate identically since we use DAGger with $\beta = 1$ for the first demonstration), combined with SA-sparse. We use the MIT/ACL open-source snap-stack³ for controlling the attitude of the MAV. The learned policy runs at 100 Hz on the onboard Nvidia Jetson TX2 (CPU), with the reference trajectory provided at 100 Hz. State estimation is from a motion capture system or onboard Visual-Inertial Odometry (VIO).

The task is to track a figure eight-shaped trajectory, with velocities up to 3.4 m/s. We evaluate the robustness of the learned policy by applying a wind-like disturbance produced by an array of 3 leaf blowers (Fig. 6). The given position reference and the corresponding trajectory are shown in Fig. 6a. The effects of the wind disturbances are clearly visible in the altitude errors and changes in commanded thrust in Fig. 6a (at $t = 11$ s and $t = 23$ s). These experiments show that the learned policy can robustly track the desired reference, withstanding challenging perturbations unseen during the training phase.

Lab2Real Transfer. We evaluate the ability of SA to learn from a single demonstration collected on a real robot in a controlled environment (lab) and generalize to previously unseen disturbances (real). We do so by using a RTMPC demonstration of a circular trajectory (velocity up to 3.5 m/s, with tight position constraints) with the multirotor, augmenting the collected demonstration with SA-sparse, and deploying the learned policy while we apply previously unseen disturbances (drag board, slung load). As shown in the sequence in Fig. 7, despite the large distribution shifts in velocity, the policy reproduces the expert demonstration and it is robust to previously unseen disturbances. Our video⁴ shows more examples.

Experimental Comparison. Table III reports a real-world comparison of SA (one demonstration) with MPC, RTMPC, and DAGger+DR (10 or 20 demonstrations from RTMPC) on the task of tracking different trajectories with a duration of 27 s, while the robot is subject to (1) wind speed up to 10 m/s, (2) a slung load of 250 grams, and (3) a surface attached at the bottom of the robot that produces extra drag (0.2 m²). All

³<https://gitlab.com/mit-acl/fsw/snap-stack>

⁴<https://youtu.be/-uiarBY1STU>

			Figure-8 (27 s, 3.0 m/s, w/o tight pos constr.)						Circle (27 s, 3.5 m/s, w/ tight position constraints)										Constraints satisfied
Method	Agent	MAE type (m, ↓)	# of Dem.	No Disturbance		Slung Load			# of Dem.	No Disturbance		Slung Load		Slung Load+Wind		Drag+Wind			
Expert	MPC	Tracking	n.a.	0.143	0.145	0.158	0.418		n.a.	0.151	0.183	0.207	0.563	0.193	0.506	0.219	0.284		No
	RTMPC	Tracking	n.a.	0.144	0.114	0.158	0.423		n.a.	0.270	0.161	0.309	0.561	0.291	0.433	0.338	0.287		Yes
Student	Dagger+DR	Gap from RTMPC	10	0.062	0.124	0.091	0.060		20	0.035	0.034	0.059	0.103	0.043	0.033	0.052	0.025		Yes
	SA-Sparse	Gap from RTMPC	1	0.057	0.121	0.081	0.100		1	0.037	0.034	0.067	0.090	0.036	0.040	0.060	0.033		Yes

TABLE III: Mean Absolute Error (MAE) in tracking a trajectory in experiments. *Tracking MAE* is the distance of the agent’s trajectory from the reference. *Gap from RTMPC* is the distance of the agent’s trajectory from the trajectory obtained using RTMPC (under the same type of disturbance). Results averaged across 3 Circles and 2 Figure-8 per agent.

TABLE IV: Time (ms) to compute an action for the linear RTMPC expert (L-RTMPC) and the DNN policy (Policy). **The DNN policy is 280 times faster than the optimization-based expert (onboard), and 25 times faster (offboard).** Offboard computer (numerical evaluation and training): Intel i9-10920 with two RTX 3090 GPUs. Onboard implementation (C++, optimized for speed): on NVIDIA TX2 CPU.

		Time (ms)				
Computer	Method	Setup	Mean	SD	Min	Max
Offboard	L-RTMPC	CVXPY/OSQP	4.28	0.39	4.21	16.66
	Policy	PyTorch	0.17	0.00	0.17	0.22
Onboard	L-RTMPC	C++/CVXGEN	8.4	1.4	4.5	15.9
	Policy	C++/Eigen	0.03	0.01	0.02	0.24

the policies are learned in simulation. The results confirm our numerical findings, highlighting that SA-sparse achieves better or comparable performance and robustness than Dagger+DR, but under reduced training effort (one demonstration instead of 10-20). In addition, the evaluation highlights that RTMPC achieves larger tracking errors when the position constraints are tight (e.g., the reference trajectory is close to the position constraint), due to RTMPC’s ability to maintain a safe distance from such constraints. However, this same property allows RTMPC to be safe (no constraint violation), unlike MPC which violates position constraints in the case of Slung Load + Wind. The learned policy runs at 500 Hz, while the RTMPC and MPC run at their maximum rates (100 Hz, occupying the entire CPU).

Computation. Table IV shows that the DNN policy is 280 times faster than the expert on the CPU of the onboard computer (Nvidia Jetson TX2). Note that the computational cost of a traditional linear MPC is comparable to the one of its linear RTMPC variant [35], further highlighting the computational benefits of our approach when compared to traditional MPC.

D. Numerical and Hardware Evaluation for Learning and Generalizing to Multiple Trajectories

We evaluate the ability of the proposed approach to track multiple trajectories while generalizing to unseen ones. To do so, we define a training distribution of reference trajectories (circle, position step, figure-8) and a distribution for these trajectory parameters (radius, velocity, position). During training, we sample at random a desired, 7 s long (70 steps) reference with randomly sampled parameters, collecting a demonstration and updating the proposed policy, while testing on a set of 20, 7 s long trajectories randomly sampled from the defined distributions. We monitor the robustness and performance of the different methods, with force disturbances (from $\mathbb{W}_{T_1}$) applied in the target domain. The results of the numerical evaluation, shown in Fig. 8, confirm that SA-sparse i) achieves robustness and performance comparable to the expert in a sample efficient way, requiring fewer than half the number of demonstrations needed for the baseline approaches; ii) simultaneously learns to generalize to multiple trajectories randomly sampled from the training distribution. Note that at convergence (from demonstration 20 to 30),

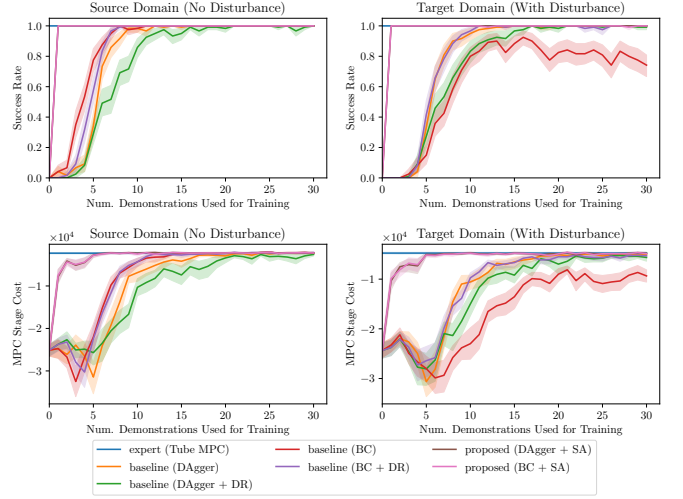


Fig. 8: Robustness (*Success Rate*) and performance (*MPC Stage Cost*) of SA (with 95% confidence interval), compared to the number of demonstrations used for training. The task is tracking previously unseen trajectories, without and with wind-like disturbances. The proposed SA-sparse strategy learns and generalize to unseen trajectories with fewer demonstrations. The lines for SA-based methods overlap. Evaluation across 20 randomly sampled trajectories per demonstration, for 6 random seeds.

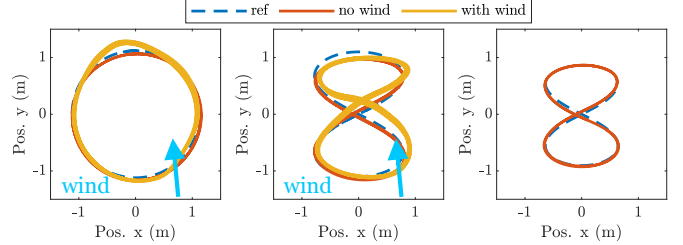


Fig. 9: Examples of different trajectories arbitrary chosen from the training distribution, and tested in hardware experiments with and without strong wind-like disturbances produced by leaf blowers. The employed policy is trained with 10 demonstrations (when other baseline methods have not fully converged yet, see Fig. 8) using Dagger+SA (sparse). This highlights that SA can learn multiple trajectories in a more sample-efficient way than other IL methods, retaining RTMPC’s robustness and performance.

Dagger+SA achieves the closest performance to the expert (2.7% *expert gap*), followed by BC+SA (3.0% *expert gap*). The hardware evaluation, performed with Dagger+SA-sparse (10 demonstrations), is shown in Fig. 9. It confirms that the obtained policy is experimentally capable of tracking multiple trajectories under real-world disturbances/model errors.

E. Extra Comparisons and Hyperparameter Study

Comparison with Other Optimal Control Approaches. SA-sparse (single demonstration) is compared in simulation with: 1) a linear trajectory tracking MPC, based on [6] (denoted **MPC**), 2) the same MPC combined with a disturbance observer (Kalman filter) that estimates online additive force disturbances, updating the model used by the MPC [6] (denoted **MPC+DO**), and 3) RTMPC (expert). The considered trajectories include position, velocity and actuation constraints, have velocities ranging in 2.0 – 3.5 m/s, and have 30 s duration each. The

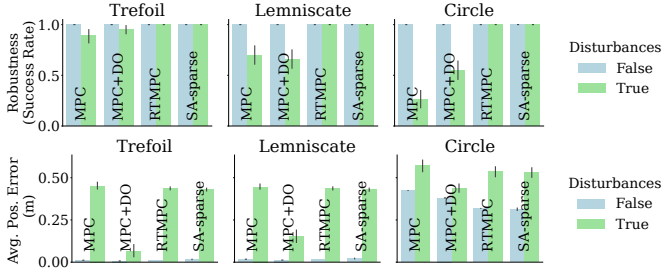


Fig. 10: Comparison of performance and robustness of a traditional MPC, MPC combined with a disturbance observer (MPC+DO), robust tube MPC (RTMPC), and the policy learned from RTMPC (SA-sparse, one demonstration). The learned policy inherits the superior robustness properties of RTMPC, while achieving comparable or better performance than MPC. We consider two scenarios, with wind disturbances (True) or without (False).

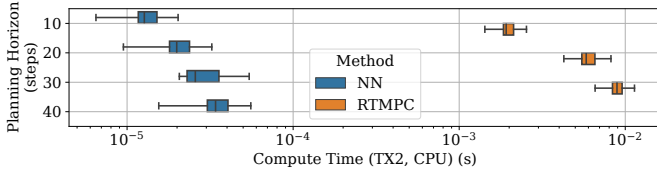


Fig. 11: Computational cost of the DNN policy (two hidden layers, 32 neurons/layer, C++) and the onboard RTMPC expert (CVXGEN, C++) as a function of the planning horizon length. The DNN achieves over 2 orders of magnitude improvement in computational efficiency.

results are presented in Fig. 10 and highlight that, while the adaptive variant of MPC (MPC+DO) achieves the lowest average tracking errors, RTMPC is superior in terms of robustness (constraint satisfaction), while the learned policy successfully inherits the robustness properties of RTMPC, with minimal trade-offs in terms of position errors.

Hyperparameter Study. First, Fig. 11 studies the effects on onboard computation when varying the planning horizon, highlighting (1) two-orders-of-magnitude improvements in the onboard computation of the DNN policy compared to the RTMPC expert, and that (2) the computational benefits of the policy increase as the planning horizon increases. Second,

TABLE V: Comparison of DNN architectures for SA-sparse policies (one demonstration). The result shows low sensitivity to the choice of DNN architecture.

NN (Neurons/Layer)	Robustness		Performance		Performance		Computation	
	Succ. Rate (%)	Pos. Error (m)	Pos. Error (m)	Expert Gap (%)	Expert Gap (%)	(ms, TX2, CPU)		
	mean	std	mean	std	mean	std	mean	std
[32, 32]	99.9	3.3	0.29	0.19	5.4	5.3	0.021	0.01
[64, 32]	100.0	0.0	0.30	0.20	4.9	4.9	0.030	0.01
[64, 64]	99.9	2.4	0.30	0.20	4.8	5.0	0.033	0.01
[64, 64, 32]	99.6	6.2	0.29	0.19	5.3	6.2	0.039	0.01
[128, 128]	99.9	3.3	0.30	0.20	4.8	5.2	0.163	0.05

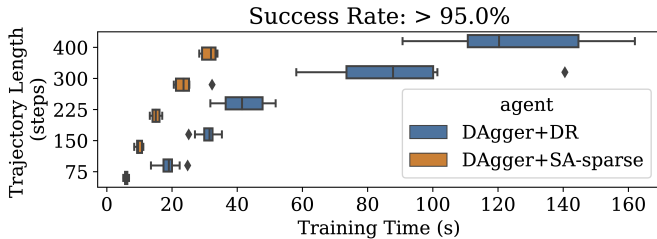


Fig. 12: Time to generate a policy (data collection in simulation and training) compared to the complexity (number of time-steps in the trajectory) of the mission to be learned. The results highlight that SA-sparse enables learning of robust policies in significantly lower time than DAgger+DR, and it scales better as the length of the task increases. Note: one step corresponds to 0.1 s. Trajectory: eight-shaped (Lemniscate) followed by a vertical circular trajectory, with velocities up to 3.5 m/s.

Table V studies robustness, performance, and onboard computation of the learned policy as a function of the size (layers, hidden neurons/layer) of DNN, highlighting that robustness and performance are minimally affected by these parameters. While onboard computation grows with the size of the network, it remains significantly lower than the onboard RTMPC expert. Last, Fig. 12 studies the time required to train a policy that achieves a success rate $> 95\%$ under wind, as a function of task complexity (length of the trajectory). This result highlights significant improvements in the scalability of our method when compared to the most robust baseline, DAgger+DR.

VII. EVALUATION - LEARNING FROM NONLINEAR RTMPC

In this Section, we evaluate the ability of our method to efficiently learn acrobatic flight maneuvers using demonstrations collected from nonlinear RTMPC.

A. Evaluation Approach

Task Description. The goal is to perform a flip, i.e., a 360° rotation about the body-frame x -axis, in near-minimum time. This is a challenging maneuver, as it covers a large nonlinear envelope of the dynamics of the Micro Aerial Vehicle (MAV), and the near-minimum time objective function, combined with the need to account for uncertainties, pushes the actuators close to their physical limits.

Simulation Environment. The simulation environment for training/numerical evaluations is the same as in Section VI, i.e., implements the nonlinear multirotor model in Section V-A. In the training domain (source, $\mathcal{S}$), $\mathbb{W}_{\mathcal{S}} = \{\emptyset\}$, while in the deployment domain (target, $\mathcal{T}$) $\mathbb{W}_{\mathcal{T}} = \{f_{\text{ext}} | 0.001mg \leq f_{\text{ext}} \leq 0.3mg\}$, sampled according to Eq. (33).

Nonlinear RTMPC. We generate a safe nominal flip trajectory using MECO-Rocket [58] and IPOPT. Because this nominal trajectory happens in the plane spanned by the orthogonal vectors defining the y and z axis of the inertial reference frame W , for simplicity, we project the dynamics onto the y and z axes, resulting in a two-dimensional model of the MAV used to generate the nominal plan. The nominal flip trajectory can therefore be obtained by setting the initial rotation around the z to be 0, and the desired final attitude to be 2π , while the remaining initial/terminal states are all set to zero.

The ancillary NMPC is solved using the SQP solver ACADOS [59], and runs in simulation at 50 Hz. Sensitivities for DA (Eq. (21)) are computed using the built-in sensitivity computation in the chosen solver, HPIPM [60]. We remark that the employed ancillary NMPC uses the full 3D multirotor model in Eq. (26), therefore performing 3D disturbance rejection – a critical requirement for real-world deployments. For a more challenging and interesting comparison to the considered IL baselines, the ancillary NMPC uses the SQP_RTI setting of ACADOS. This setting performs only a single SQP iteration per timestep, enabling significant speed-ups in the solver, and it is often employed in real-time, embedded implementations of NMPC. This setting creates an advantage, in terms of training time, to IL methods that require querying the expert multiple times (the baselines of our comparison), as it speeds-up the computation time of the expert. The other ACADOS parameters given in Table VI were chosen as they enabled higher overall performance/accuracy in the selected acrobatic maneuver. Last,

TABLE VI: Parameters for the ancillary NMPC, solved via ACADOS [59].

Parameter(s)	Value(s)
Hessian Approximation	Gauss-Newton
QP solver	Partial Condensing HPIPM [60]
NLP/QP Tolerance	$10^{-8}/10^{-8}$
Levenberg-Marquardt	10^{-4}
Integrator Type	Implicit Runge-Kutta
Max. # Iterations QP Solver	100
Horizon (N , steps)/(time, seconds)	50/1.0

we introduce a discount factor $\gamma = 0.95$ in the stage cost of Eq. (16) to aid the convergence of the solver.

Student Policy. The student is a 2-hidden layers, fully connected DNN with $\{64, 32\}$ neurons/layer, and ReLU activation functions. The input has dimension 14, as it contains the current state ($n_x = 10$), time t , and a desired final position p^{des} (fixed to the origin). To simplify the learning and DA procedure, we enforce continuity to the quaternion input of the policy via [61, Eq. 3], avoiding the need to increase the training data/demonstrations at every timestep to account for the fact that q and $-q$ encode the same orientation.

Baselines and Evaluation Metrics The baselines match those in Section VI (Dagger, BC and their combination with DR, DA- N_s (linear interpolation) and DA- N_s (expert neighborhood), generating N_s samples per timestep. The monitored metrics (*Robustness*, *Performance* and *Training Time*) match those in Section VI, with the difference that performance is based on the stage cost of the ancillary NMPC Eq. (16).

Training Details. As in Section VI, training is performed by collecting demonstrations with the multirotor starting from slightly different initial states inside the tube centered around the origin. The nominal flip maneuver is pre-generated, as the goal state $x_{t_0}^*$ (with $t_0 = 0$) does not change, and only the ancillary NMPC is solved at every timestep. The resulting flip maneuver takes about $T_f = 2.5$ s, and demonstrations are collected over an episode of length 3.0 s, at 50 Hz ($T = 150$ environment steps per demonstration). **Data collection.** For SA-methods, we collect demonstrations one-by-one, and we implement the fine-tuning procedure described in Section IV-C by performing DA with the first collected demonstration, while we do not perform DA for the following demonstrations. Because of its computational efficiency, we always use the sensitivity-based DA (i.e., Eq. (19), assuming no changes in active set of constraints). In addition, to study the effects of varying the number of samples used for DA, we introduce SA- N_s , a variant of SA where we sample uniformly inside the tube $N_s = \{25, 50, 100\}$ samples for every timestep. For the baselines, in order to speed-up the demonstration collection phase, and thereby avoid excessive re-training of the policy, we collect demonstrations in batches of 10, for 20 batches. An exception is made for DA- N_s (expert neighborhood) where, similar to SA, we collect demonstrations one-by-one to better study its sample efficiency, and the corresponding actions are obtained using the same sensitivity-based approximation of the ancillary NMPC employed by SA. Therefore, DA- N_s (expert neighborhood) constitutes an ablation of SA- N_s , where sampling is restricted to a smaller volume than the entire tube. **Evaluation** Each time the policy is updated, we evaluate it 20 times in source $\mathcal{S}$ and target $\mathcal{T}$ environments. The evaluations are repeated across 10 random seeds. To further speed-up training of all the methods, we update the previously trained policy using only the newly collected batch of demonstrations

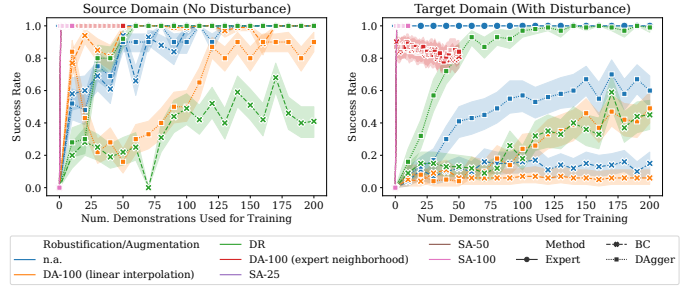


Fig. 13: Robustness as a function of the number of training demonstrations. The proposed SA-methods overlap on the top-left part of the diagram, achieving full success rate in both the environment without and with wind-like disturbances.

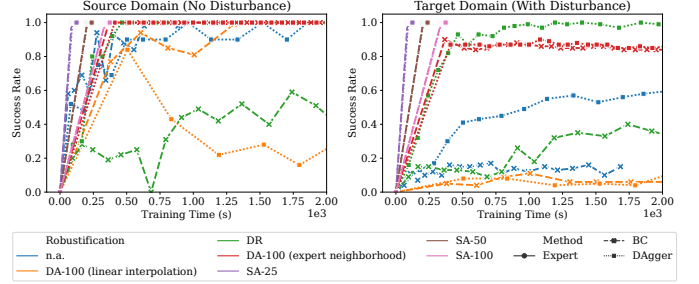


Fig. 14: Robustness as a function of the training time. SA-methods achieve full robustness under uncertainties in a fraction of the training time required by the best performing robust baseline, DAgger + DR.

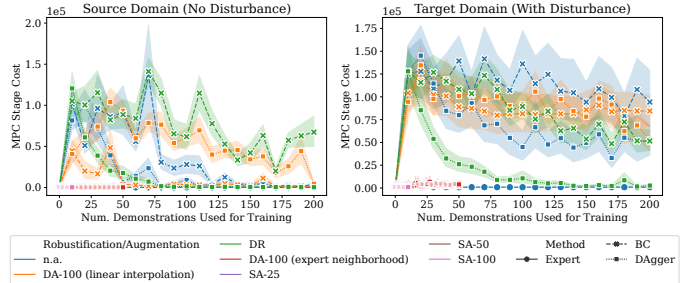


Fig. 15: Performance as a function of the number of training demonstrations. SA-methods achieve performance close to the expert in less than 10 demonstrations. The best-performing baseline, DA-100 (expert neighborhood), achieves comparable performance, but is not robust when subject to wind disturbances, as shown in Fig. 13.

(or single demonstration, for SA). All the policies are trained using the ADAM optimizer for up to 400 epochs, but we terminate training if the validation loss (from 30% of the data) does not decrease within 30 epochs.

B. Numerical Evaluation: Robustness, Performance, Efficiency Comparison with Baselines. We start by evaluating the robustness and performance of the proposed approach as a function of the number of demonstrations collected in simulation, and as a function of the training time.

Fig. 13 shows the robustness of the considered method as a function of the number of expert demonstrations. It reveals that SA-based approaches can achieve full success rate in the environment with disturbances (target, $\mathcal{T}$) and without disturbances (source, $\mathcal{S}$) after a single demonstration, while the best-performing baseline, DAgger+DR, requires about 60 demonstrations to achieve full robustness in $\mathcal{S}$, and more than 100 in $\mathcal{T}$. SA-based methods, therefore, enable more than one order of magnitude reduction in the number of demonstrations (interactions with the environment) compared to DAgger+DR. As previously observed in Section VI, DAgger alone is not robust. Additionally, BC methods fail to converge, potentially

Method	Robustification/ Augmentation	# of Demonstr.	Robustness success rate (%, $\uparrow$)				Performance expert gap (%, $\downarrow$)				Efficiency training time (s, $\downarrow$)			
			S		T		S		T		S		T	
			mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
BC	DA-100 (expert neighborhood)	1	100	0	90	30	9	6	562	1576	367	87		
		2	100	0	85	36	6	4	312	671	512	90		
		10	100	0	86	35	5	4	462	2932	1166	135		
		50	100	0	81	39	5	2	323	838	4629	185		
		100	100	0	87	34	12	8	562	2375	409	90		
DAgger	DA-100 (expert neighborhood)	1	100	0	87	34	9	10	425	1478	520	97		
		2	100	0	89	31	4	3	187	354	1149	107		
		10	100	0	84	37	3	2	440	1623	4608	141		
		50	100	0	92	27	82	39	9094	20608	3096	5497	392	34
		100	100	0	97	10	634	711	1277	4947	810	104		
DAgger	DR	1	100	0	99	10	91	73	274	1247	1970	154		
		2	100	0	100	0	553	462	211	228	84	9		
		10	100	0	97	17	41	39	371	1808	88	10		
		50	100	0	97	17	33	42	226	815	115	10		
		100	100	0	100	0	107	175	90	83	117	14		
BC	SA-sparse (18)	1	100	0	100	0	956	402	270	384	87	15		
		2	100	0	100	0	148	140	107	150	90	14		
		10	100	0	100	0	107	175	90	83	117	14		
		50	100	0	100	0	421	193	105	116	204	32		
		100	100	0	100	0	76	31	66	56	207	31		
SA-25	1	1	100	0	100	0	55	28	76	102	235	31		
		2	100	0	100	0	291	154	89	105	339	72		
		10	100	0	100	0	57	20	76	85	342	72		
		50	100	0	100	0	33	21	97	118	369	72		
		100	100	0	100	0	747	705	319	879	85	6		
DAgger	SA-sparse (18)	1	100	0	100	0	222	122	142	219	89	6		
		2	100	0	100	0	29	24	114	150	117	6		
		10	100	0	100	0	579	224	160	168	92	12		
		50	100	0	100	0	366	279	122	182	96	12		
		100	100	0	100	0	110	115	100	120	124	12		
SA-50	1	1	100	0	100	0	361	161	78	91	206	28		
		2	100	0	100	0	169	117	77	80	210	28		
		10	100	0	100	0	56	77	82	107	237	28		
		50	100	0	100	0	309	133	92	105	342	29		
		100	100	0	100	0	100	61	77	96	346	29		
SA-100	1	1	100	0	100	0	30	28	90	109	373	29		
		2	100	0	100	0								
		10	100	0	100	0								
		50	100	0	100	0								
		100	100	0	100	0								

TABLE VII: Performance, robustness and training time for SA-based methods after 1, 2, and 10 demonstrations, compared with the best performing baselines, DAgger+DR and sampling in the expert neighborhood (DA-100 (expert neighborhood)), in the environment without wind disturbances (S , source), and with (T , target). Robustness is color-coded from white (100%) to red (90% or below). Performance and training time are color-coded from green (fast training time, small expert gap) to red (long training time, large expert gap). The results highlight that SA-methods achieve high robustness and close to expert performance compared to DAgger+DR, even after a single demonstration, and their performance can be further improved via additional fine-tuning demonstrations. Methods based on DA-100 (expert neighborhood) are not robust and struggle to achieve high performance (low expert gap) in the target domain. We note that DAgger and BC-based approaches differ at one demonstration due to non-determinism in the training procedure.

due to the lack of sufficiently meaningful exploration and the forgetting caused by the iterative training strategy employed. In addition, DA (expert neighborhood) confirms the importance of using the tube as a support of the sampling distribution, as the method achieves robustness and demonstration efficiency in the source domain, but fails to achieve robustness in the target domain, unlike SA. DA (linear interpolation) offers an initial boost in demonstration efficiency but struggles to achieve high robustness even within the source domain. This may be due to the introduction of far-from optimal actions.

Fig. 14 additionally shows the robustness as a function of the training time (recall, this includes demonstration collection and policy train). The results show that the demonstration-efficiency of SA-based methods translates into significant improvements in training time, as DAgger+DR requires more than 3 times the training time than SA-based approaches. These improvements are larger for the variants of SA that generate fewer extra samples (e.g., SA-25).

Last, Fig. 15 reports the *performance* as a function of the number of demonstrations. The results indicate that SA-based methods can achieve low tracking errors even after a single demonstration. Furthermore, employing a fine-tuning phase (after the initial demonstration) proves highly advantageous in further reducing this error, thereby reducing the performance gap between policies obtained via SA and the expert.

Comparison of Sampling Strategies. Table VII provides a detailed comparison of performance, robustness, and training time of the different variants of SA methods, as a function of the number of demonstrations (1, 2 and 10), and compares those with the best-performing baselines, DAgger+DR, and methods

TABLE VIII: Time (ms) to compute a new action for the ancillary NMPC, the safe planner of the nonlinear RTMPC expert (N-RTMPC) and the proposed DNN policy (Policy). **The policy is 180 times faster than the NMPC in [39] (on the same onboard computer).** The onboard CPU is an Intel i9-10920, onboard is an NVIDIA Jetson TX2 (CPU). Note that the faster inference time than the linear case is caused by the input dimension being smaller (14 vs 188).

CPU	Method	Setup	Time (ms)			
			Mean	SD	Min	Max
Offboard	N-RTMPC, ancillary NMPC	ACADOS [59]	7.28	0.15	7.05	8.00
	N-RTMPC, safe plan	IPOPT	5812	226	4828	6010
	Policy	PyTorch	0.11	0.01	0.11	0.27
Onboard	NMPC (from [39, Fig. 17])	ACADO [62]	2.7	n.a.	n.a.	n.a.
	Policy	C++/Eigen	0.015	0.005	0.006	0.101

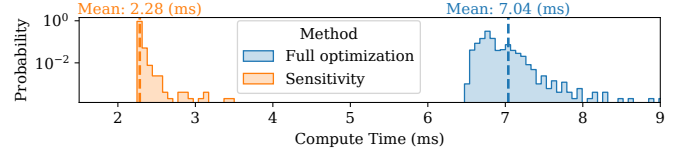


Fig. 16: Distribution of the time (ms) to solve the full optimization problem for the ancillary NMPC (Eq. (16)), and to additionally compute the sensitivity matrix (Eq. (21)). The sensitivity matrix require only 32% of the time to solve the full optimization problem. Analysis performed on a Intel i9-10920 using ACADOS with settings in Table VI. Note the log scale of the y axis.

based on sampling in the expert neighborhood. As expected, SA methods that require fewer samples obtain significant improvements in training time compared to DAgger+DR, while increasing the number of samples is beneficial in reducing the mean and the variance of the expert gap, both with and without disturbances, while DA (expert neighborhood) struggles to achieve high robustness and low expert gap in the target domain, despite the large number of samples used per timestep (100). Table VII additionally highlights the benefits of fine-tuning, as even methods that use few samples (e.g., SA-sparse, SA-25) can obtain a significant performance improvement after a single fine-tuning demonstration (2 demonstrations in total), while there are diminishing returns for additional fine-tuning demonstrations (e.g., 10 demonstrations). In addition, in the data-sparse regime (e.g., SA-18), using DAgger for fine-tuning appears more beneficial than BC.

Computation. The computation time is reported in Table VIII, highlighting that the average computation time of the policy on the onboard Nvidia Jetson TX2 is 0.015 ms, an 180-fold improvement compared to the value reported in [63] for a state-of-the-art NMPC for quadrotors. In addition, Fig. 16 reports the time to compute the sensitivity matrix, highlighting that it requires on average 2.28 ms, about only 32% of the time required to solve the full optimization problem. More importantly, this matrix is computed only once per timestep and can be used to draw many samples at small additional cost (equivalent to solving a vector-matrix multiplication) instead of solving the full optimization problem for each sample. The average time to step the training environment is 2.1 ms.

C. Hardware Evaluation

We experimentally evaluate the ability of the policy to perform a flip on a real multirotor, under real-world uncertainties such as model errors (e.g., inaccurate thrust to battery voltage mappings, aerodynamic coefficients, moments of inertia) and external disturbances (e.g., ground effect). The tested policy is obtained using DAgger+SA-25 trained after 2 demonstrations (the first with DA, the second for fine-tuning), as the method represents a good trade-off between performance, robustness

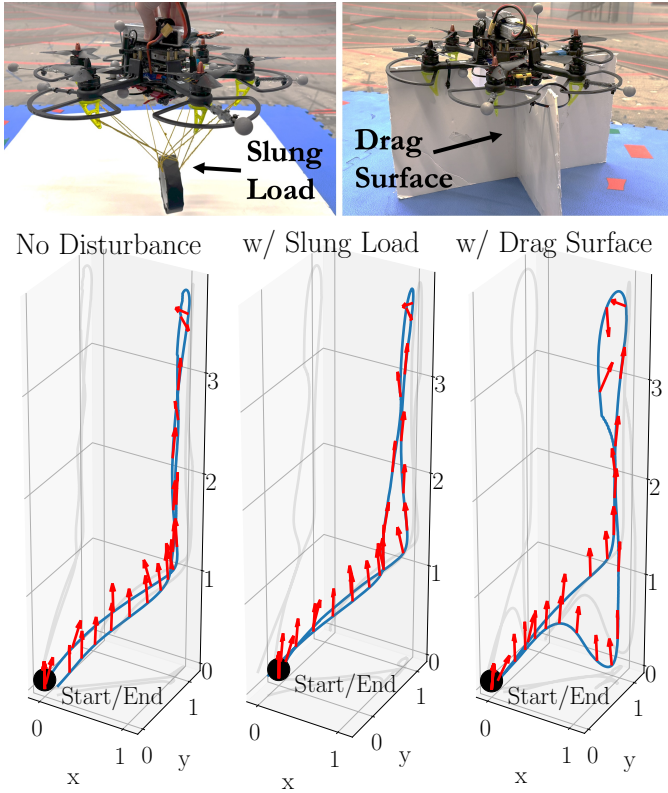


Fig. 17: Aerobatic (flip) flight in experiments, using a policy learned from a nonlinear Robust Tube MPC in about 100 s of data collection (in sim., on a single CPU) and training time. The policy runs onboard (TX2, CPU, at up to 500 Hz, average inference time 0.015 ms) and is robust to disturbances (slung load of 0.18 Kg., drag surface of 0.2 m² and 0.13 Kg). Red arrows denote the direction of the thrust vector, showing that the flip occurs at the point of highest altitude. Units in (m).

and training time. As in Section VI, we deploy the learned policy on an onboard Nvidia Jetson TX2, where it runs at 100 Hz. The maneuver includes a take-off/landing phase consisting of a 1 m ramp on x - y - z in W and overall has a total duration of 6 s. The maneuver is repeated 5 times in a row, to demonstrate repeatability, recording successful execution of the maneuver and successful landing at the designated location in all the cases. Fig. 1 shows a time-lapse of the different phases of the maneuver (excluding the ramp from and to the landing location). The 3D position of the robot, as well as the direction of its thrust vector, are shown for two runs in Fig. 17, highlighting the large distance and altitude traveled in a short time. Fig. 18 additionally shows some critical parameters of the maneuvers, such as the attitude and the angular velocity, as well as thrust and the vertical velocities. It highlights that the robot rotates at up to 11 rad/s, and the overall 360° rotation takes about 0.5 s. In addition, the maneuver is repeated under even more challenging uncertainties, obtained by attaching either (a) a slung-load or (b) a drag surface to the robot, as shown in Fig. 17, deploy the policy onboard at 500 Hz. The experiment is repeated 3 consecutive times per disturbances, achieving 100% success rate, and a resulting trajectory for each disturbance is shown in Fig. 17⁵. Overall, these results validate our numerical analysis and highlight the robustness and performance of a policy efficiently trained from 2 demonstrations and about

⁵Note that the gains of the cascaded attitude controller were increased compared to the scenario without extra disturbances. This was done to account for the fact that the attitude controller is not explicitly robust/adaptive to these new uncertainties, unlike the learned policy

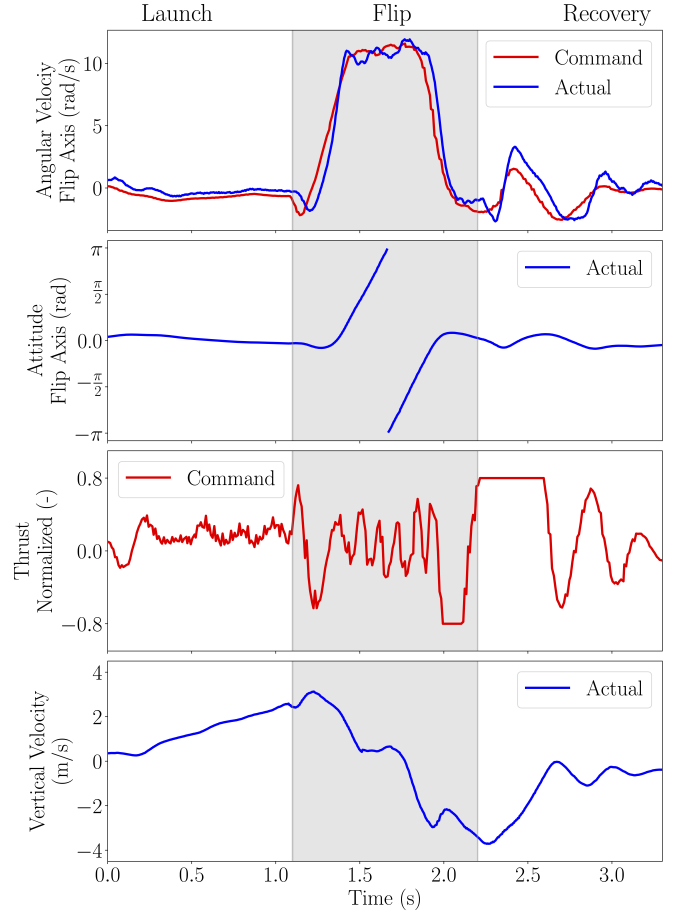


Fig. 18: Control inputs and relevant states during the real-world acrobatic flip maneuver. Despite the large level of uncertainties (inaccurate thrust-to-battery voltage mappings, hard-to-model aerodynamic effects), that require the usage of the maximum thrust allowed, the maneuver is completed successfully, performing a flip with an angular velocity of about 11 rad/s. Note that the actual thrust t_{cmd} can be related to the normalized thrust $\bar{t}_{\text{cmd}}$ via $t_{\text{cmd}} = mg(1 + \bar{t}_{\text{cmd}})$, where mg is the weight force of the robot.

100 s of training time. Our video submission [64] includes an additional experiment demonstrating near-minimum time navigation from one position to another, starting and ending with velocity close to zero, using a policy trained with two demonstrations (Dagger+SA-25).

VIII. DISCUSSION, LIMITATIONS AND FUTURE WORK

This work has demonstrated that it is possible to generate policies from MPC that are fast and robust in the real world, while requiring (a) few queries to the expensive controller, (b) few environment interactions, (c) and short training times. Evaluations have validated the performance and data/computation efficiency, additionally showing that increasing the number of samples in DA or introducing a fine-tuning procedure can further improve performance. These findings have broad applicability beyond the MPC and IL communities. For example, our method can serve as an efficient policy pre-training procedure, using model and uncertainty priors, for subsequent fine-tuning via model-free Reinforcement Learning (RL), reducing inefficient random exploration in RL or simplifying reward design.

We acknowledge some limitations, which open many exciting opportunities for future work. First, while our methodology has demonstrated real-world robustness, in the future we would

like to leverage DNN reachability tools [65]–[67] to provide robustness certificates, enabling the deployment on safety-critical systems. Second, while easy-to-compute fixed-size approximations of the tube have been sufficient to guide our DA strategy, future work will focus on leveraging tubes with varying cross-sections, enabling even more aggressive expert demonstrations. Third, while our approach showed robustness to small uncertainties in the rotational dynamics, we aim to combine our method with adaptive variants of the cascaded attitude controllers to avoid tuning the attitude controller under large uncertainties in the rotational dynamics. Additionally, we would like to exploit the training efficiency of our approach to design adaptation strategies for trajectory tracking, for example by quickly generating new policies once new estimates of the model/environment become available. Last, we would like to leverage the efficiency and robustness of the obtained policies on aerial platforms with extreme payload/compute constraints [10], [11].

IX. CONCLUSION

This work has presented an IL strategy to efficiently train a robust DNN policy from MPC. Key ideas were to (a) leverage a Robust Tube variant of MPC, called RTMPC, to collect demonstrations using existing IL methods (Dagger, BC), and (b) augment the collected demonstrations with *efficiently-generated* extra state-and-actions samples *from the tube of the controller*, an approximation of the support of the state distribution that the learned policy will encounter when subject to uncertainties. While the linear ancillary controller in linear RTMPC provides extra data in a computationally efficient way, as shown in our conference paper [47], the same efficiency can be challenging to achieve when leveraging nonlinear variants of RTMPC [36]. Therefore, in this journal extension of [47] we have presented a strategy to efficiently perform tube-guided DA leveraging a sensitivity-based approximation of the ancillary controller in NMPC and using a fine-tuning phase to reduce the errors caused by these approximations. Experimental evaluations on a multirotor have validated our numerical findings of efficiency and robustness, showing that a policy trained in only 100 s can perform a flip under uncertainties, while requiring only 15 μ s to compute commands onboard.

REFERENCES

- [1] F. Borrelli, A. Bemporad, and M. Morari, *Predictive control for linear and hybrid systems*. Cambridge University Press, 2017.
- [2] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model predictive control: theory, computation, and design*. Nob Hill Publishing Madison, WI, 2017, vol. 2.
- [3] B. T. Lopez, J.-J. E. Slotine, and J. P. How, “Dynamic tube MPC for nonlinear systems,” in *2019 American Control Conference (ACC)*. IEEE, 2019, pp. 1655–1662.
- [4] B. T. Lopez, “Adaptive robust model predictive control for nonlinear systems,” Ph.D. dissertation, Massachusetts Institute of Technology, 2019.
- [5] W. Li and E. Todorov, “Iterative linear quadratic regulator design for nonlinear biological movement systems,” in *ICINCO (1)*. Citeseer, 2004, pp. 222–229.
- [6] M. Kamel, M. Burri, and R. Siegwart, “Linear vs nonlinear MPC for trajectory tracking applied to rotary wing micro aerial vehicles,” *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 3463–3469, 2017.
- [7] M. V. Minniti, F. Farshidian, R. Grandia, and M. Hutter, “Whole-body MPC for a dynamically stable mobile manipulator,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3687–3694, 2019.
- [8] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Aggressive driving with model predictive path integral control,” in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 1433–1440.
- [9] P. Kumar, J. B. Rawlings, and S. J. Wright, “Industrial, large-scale model predictive control with structured neural networks,” *Computers & chemical engineering*, vol. 150, 2021-07.
- [10] Y. Chen, S. Xu, Z. Ren, and P. Chirarattananon, “Collision resilient insect-scale soft-actuated aerial robots with high agility,” *IEEE Transactions on Robotics*, vol. 37, no. 5, pp. 1752–1764, 2021.
- [11] W. Giernacki, M. Skwirczyński, W. Witwicki, P. Wroński, and P. Kozierski, “Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering,” in *2017 22nd International Conference on Methods and Models in Automation and Robotics (MMAR)*. IEEE, 2017, pp. 37–42.
- [12] E. Kaufmann, A. Loquercio, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, “Deep drone acrobatics,” *Robotics, Science, and Systems (RSS)*, 2020.
- [13] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, J. A. Bagnell, and M. Hebert, “Learning monocular reactive uav control in cluttered natural environments,” in *2013 IEEE international conference on robotics and automation*. IEEE, 2013, pp. 1765–1772.
- [14] A. Reske, J. Carius, Y. Ma, F. Farshidian, and M. Hutter, “Imitation learning from MPC for quadrupedal multi-gait control,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 5014–5020.
- [15] D. A. Pomerleau, “Alvin: An autonomous land vehicle in a neural network,” Carnegie-Mellon Univ Pittsburgh PA Artificial Intelligence and Psychology, Tech. Rep., 1989.
- [16] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters, “An algorithmic perspective on imitation learning,” *arXiv preprint arXiv:1811.06711*, 2018.
- [17] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang *et al.*, “End to end learning for self-driving cars,” *arXiv preprint arXiv:1604.07316*, 2016.
- [18] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [19] J. G. Van Antwerp and R. D. Braatz, “Model predictive control of large scale processes,” *Journal of Process Control*, vol. 10, no. 1, pp. 1–8, 2000. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0959152499000505>
- [20] J. Wang, C. L. Swartz, and K. Huang, “Deep learning-based model predictive control for real-time supply chain optimization,” *Journal of Process Control*, vol. 129, p. 103049, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0959152423001361>
- [21] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 3803–3810.
- [22] A. Loquercio, E. Kaufmann, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, “Deep drone racing: From simulation to reality with domain randomization,” *IEEE Transactions on Robotics*, vol. 36, no. 1, pp. 1–14, 2019.
- [23] S. Levine and V. Koltun, “Guided policy search,” in *International conference on machine learning*. PMLR, 2013, pp. 1–9.
- [24] D. Krishnamoorthy, “A sensitivity-based data augmentation framework for model predictive control policy approximation,” *IEEE Transactions on Automatic Control*, vol. 67, no. 11, pp. 6090–6097, 2022.
- [25] —, “An improved data augmentation scheme for model predictive control policy approximation,” *arXiv preprint arXiv:2303.05607*, 2023.
- [26] S. W. Chen, T. Wang, N. Atanasov, V. Kumar, and M. Morari, “Large scale model predictive control with neural networks and primal active sets,” *Automatica*, vol. 135, p. 109947, 2022.
- [27] Y. Pan, C.-A. Cheng, K. Saigol, K. Lee, X. Yan, E. A. Theodorou, and B. Boots, “Imitation learning for agile autonomous driving,” *The International Journal of Robotics Research*, vol. 39, no. 2-3, pp. 286–302, 2020.
- [28] D. H. Jacobson and D. Q. Mayne, *Differential dynamic programming*. Elsevier Publishing Company, 1970, no. 24.
- [29] G. Kahn, T. Zhang, S. Levine, and P. Abbeel, “Plato: Policy learning using adaptive trajectory optimization,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 3342–3349.
- [30] J. Carius, F. Farshidian, and M. Hutter, “MPC-net: A first principles guided policy search,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2897–2904, 2020.

- [31] H. Tsukamoto and S.-J. Chung, "Learning-based robust motion planning with guaranteed stability: A contraction theory approach," *IEEE Robotics and Automation Letters*, 2021.
- [32] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox, "Closing the sim-to-real loop: Adapting simulation randomization with real world experience," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8973–8979.
- [33] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "Dart: Noise injection for robust imitation learning," in *Conference on robot learning*. PMLR, 2017, pp. 143–156.
- [34] T. Zhang, G. Kahn, S. Levine, and P. Abbeel, "Learning deep control policies for autonomous aerial vehicles with MPC-guided policy search," in *2016 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2016, pp. 528–535.
- [35] D. Q. Mayne, M. M. Seron, and S. Raković, "Robust model predictive control of constrained linear systems with bounded disturbances," *Automatica*, vol. 41, no. 2, pp. 219–224, 2005.
- [36] D. Q. Mayne, E. C. Kerrigan, E. Van Wyk, and P. Falugi, "Tube-based robust nonlinear model predictive control," *International journal of robust and nonlinear control*, vol. 21, no. 11, pp. 1341–1353, 2011.
- [37] H. Tsukamoto and S.-J. Chung, "Neural contraction metrics for robust estimation and control: A convex optimization approach," *IEEE Control Systems Letters*, vol. 5, no. 1, pp. 211–216, 2020.
- [38] H. Nguyen, M. Kamel, K. Alexis, and R. Siegwart, "Model predictive control for micro aerial vehicles: A survey," in *2021 European Control Conference (ECC)*. IEEE, 2021, pp. 1556–1563.
- [39] S. Sun, A. Romero, P. Foehn, E. Kaufmann, and D. Scaramuzza, "A comparative study of nonlinear mpc and differential-flatness-based control for quadrotor agile flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3357–3373, 2022.
- [40] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *Science Robotics*, vol. 6, no. 56, p. eabh1221, 2021.
- [41] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3340–3356, 2022.
- [42] T. Manzoor, H. Pei, Z. Sun, and Z. Cheng, "Model predictive control technique for ducted fan aerial vehicles using physics-informed machine learning," *Drones*, vol. 7, no. 1, p. 4, 2022.
- [43] E. Kaiser, J. N. Kutz, and S. L. Brunton, "Sparse identification of nonlinear dynamics for model predictive control in the low-data limit," *Proceedings of the Royal Society A*, vol. 474, no. 2219, p. 20180335, 2018.
- [44] A. Saviolo, G. Li, and G. Loianno, "Physics-inspired temporal learning of quadrotor dynamics for accurate model predictive trajectory tracking," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10256–10263, 2022.
- [45] N. A. Spielberg, M. Brown, and J. C. Gerdes, "Neural network model predictive motion control applied to automated driving with unknown friction," *IEEE Transactions on Control Systems Technology*, vol. 30, no. 5, pp. 1934–1945, 2021.
- [46] D. Hanover, P. Foehn, S. Sun, E. Kaufmann, and D. Scaramuzza, "Performance, precision, and payloads: Adaptive nonlinear mpc for quadrotors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 690–697, 2022.
- [47] A. Tagliabue, D.-K. Kim, M. Everett, and J. P. How, "Demonstration-efficient guided policy search via imitation of robust tube MPC," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 462–468.
- [48] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [49] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: an operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020. [Online]. Available: <https://doi.org/10.1007/s12532-020-00179-2>
- [50] K. J. Åström and R. M. Murray, *Feedback systems: an introduction for scientists and engineers*. Princeton university press, 2021.
- [51] M. Kamel, T. Stastny, K. Alexis, and R. Siegwart, "Model predictive control for trajectory tracking of unmanned aerial vehicles using robot operating system," in *Robot operating system (ROS)*. Springer, 2017, pp. 3–39.
- [52] D. D. Fan, A.-a. Agha-mohammadi, and E. A. Theodorou, "Deep learning tubes for tube MPC," *arXiv preprint arXiv:2002.01587*, 2020.
- [53] D. Limón, I. Alvarado, T. Alamo, and E. F. Camacho, "Robust tube-based mpc for tracking of constrained linear systems with additive disturbances," *Journal of Process Control*, vol. 20, no. 3, pp. 248–260, 2010.
- [54] W. M. Kouw and M. Loog, "An introduction to domain adaptation and transfer learning," *arXiv preprint arXiv:1812.11806*, 2018.
- [55] T. Lee, "Geometric tracking control of the attitude dynamics of a rigid body on $so(3)$," in *Proceedings of the 2011 American Control Conference*, 2011, pp. 1200–1205.
- [56] M. D. Shuster *et al.*, "A survey of attitude representations," *Navigation*, vol. 8, no. 9, pp. 439–517, 1993.
- [57] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [58] J. Gillis, B. Vandewal, G. Pipeleers, and J. Swevers, "Effortless modeling of optimal control problems with rokit," in *39th Benelux Meeting on Systems and Control, Date: 2020/03/10-2020/03/12, Location: Elspeet, The Netherlands*, 2020.
- [59] R. Verschuere, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados – a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, Oct 2021. [Online]. Available: <https://doi.org/10.1007/s12532-021-00208-8>
- [60] G. Frison and M. Diehl, "Hpipm: a high-performance quadratic programming framework for model predictive control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6563–6569, 2020.
- [61] T. Kusaka and T. Tanaka, "Stateful rotor for continuity of quaternion and fast sensor fusion algorithm using 9-axis sensors," *Sensors*, vol. 22, no. 20, p. 7989, 2022.
- [62] B. Houska, H. Ferreau, and M. Diehl, "ACADO Toolkit – An Open Source Framework for Automatic Control and Dynamic Optimization," *Optimal Control Applications and Methods*, vol. 32, no. 3, pp. 298–312, 2011.
- [63] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, vol. 8, no. 82, p. eadg1462, 2023.
- [64] [Online]. Available: <https://youtu.be/aWRuvy3LviI>
- [65] M. Everett, G. Habibi, C. Sun, and J. P. How, "Reachability analysis of neural feedback loops," *IEEE Access*, vol. 9, pp. 163 938–163 953, 2021.
- [66] N. Rober, S. M. Katz, C. Sidrane, E. Yel, M. Everett, M. J. Kochenderfer, and J. P. How, "Backward reachability analysis of neural feedback loops: Techniques for linear and nonlinear systems," *IEEE Open Journal of Control Systems*, vol. 2, pp. 108–124, 2023.
- [67] C. Sidrane, A. Maleki, A. Irfan, and M. J. Kochenderfer, "Overt: An algorithm for safety verification of neural network control policies for nonlinear systems," *The Journal of Machine Learning Research*, vol. 23, no. 1, pp. 5090–5134, 2022.



Andrea Tagliabue received the B.Sc. degree in automation engineering from Politecnico di Milano, Milano, Italy (2015), the M.Sc. degree in robotics, systems, and control from ETH Zurich, Zurich, Switzerland (2018), and a Ph.D. degree in aeronautics and astronautics from the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA (2024). Dr. Tagliabue was a visiting researcher at U.C. Berkeley, Berkeley, CA, USA (2017–2018), and an Engineer Affiliate with NASA's Jet Propulsion Laboratory, Pasadena, CA, USA (2018–2019). Dr. Tagliabue's

interests include learning, perception and control for agile systems. His work was awarded finalist for the Best Paper in Dynamics and Control at ICRA 2023.



Jonathan P. How (Fellow, IEEE) received the B.A.Sc. degree in aerospace from the University of Toronto, Toronto, Canada, in 1987, and the S.M. and Ph.D. degrees in aeronautics and astronautics from the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, in 1990 and 1993, respectively. Prior to joining MIT in 2000, he was an Assistant Professor with Stanford University, Stanford, CA, USA. He is currently the Richard C. Maclaurin Professor of aeronautics and astronautics at MIT. Dr. How's awards include the IEEE CSS

Distinguished Member Award (2020), AIAA Intelligent Systems Award (2020), IROS Best Paper Award on Cognitive Robotics (2019), and the AIAA Best Paper in Conference Awards (2011, 2012, 2013). He was the Editor-in-Chief of IEEE Control Systems Magazine (2015–2019), is a Fellow of AIAA, and was elected to the National Academy of Engineering in 2021.